A Semisupervised Siamese Network for Hyperspectral Image Classification

Sen Jia, *Senior Member, IEEE*, Shuguo Jiang, Zhijie Lin, Meng Xu, *Member, IEEE*, Weiwei Sun, *Senior Member, IEEE*, Qiang Huang, Jiasong Zhu, and Xiuping Jia, *Fellow, IEEE*

***Abstract*— With the development of hyperspectral imaging technology, hyperspectral images (HSIs) have become important when analyzing the class of ground objects. In recent years, benefiting from the massive labeled data, deep learning has achieved a series of breakthroughs in many fields of research. However, labeling HSIs requires sufficient domain knowledge and is time-consuming and laborious. Thus, how to apply deep learning effectively to small labeled samples is an important topic of research in HSI classification. To solve this problem, we propose a semisupervised Siamese network that embeds Siamese network into a semisupervised learning scheme. It integrates an autoencoder module and a Siamese network to, respectively, investigate information in a large amount of unlabeled data and rectify it with a limited labeled sample set, which is called 3DAES. First, the autoencoder method is trained on the massive unlabeled data to learn the refinement representation, creating an unsupervised feature. Second, based on this unsupervised feature, limited labeled samples are used to train a Siamese network to rectify the unsupervised feature to improve feature separability among various classes. Furthermore, by training the Siamese network, a random sampling scheme is used to accelerate training and avoid imbalance among various sample classes. Experiments on three benchmark HSI datasets consistently demonstrate the effectiveness and robustness of the proposed 3DAES approach with limited labeled samples. For study replication, the code developed for this study is available at https://github.com/ShuGuoJ/3DAES.git.**

Manuscript received August 18, 2021; accepted September 20, 2021. Date of publication October 8, 2021; date of current version January 26, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 41971300, Grant 61901278, and Grant 62001303; in part by the Key Project of Department of Education of Guangdong Province under Grant 2020ZDZX3045; in part by the Natural Science Foundation of Guangdong Province under Grant 2021A1515011413; in part by China Postdoctoral Science Foundation under Grant 2021M692162; and in part by Shenzhen Scientific Research and Development Funding Program under Grant 20200803152531004. *(Corresponding authors: Meng Xu; Qiang Huang.)*

Sen Jia, Shuguo Jiang, Zhijie Lin, Meng Xu, and Qiang Huang are with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China, and also with the Key Laboratory for Geo-Environmental Monitoring of Coastal Zone of the Ministry of Natural Resources, Shenzhen University, Shenzhen 518060, China (e-mail: senjia@szu.edu.cn; 2462230948@qq.com; 18814103637@163.com; m.xu@szu.edu.cn; jameshq@szu.edu.cn).

Weiwei Sun is with the College of Architectural Engineering, Civil Engineering and Environment, Ningbo University, Ningbo 315211, China (e-mail: nbsww@outlook.com).

Jiasong Zhu is with the Key Laboratory for Geo-Environmental Monitoring of Coastal Zone of the Ministry of Natural Resources, Shenzhen University, Shenzhen 518060, China (e-mail: zhujiasong@gmail.com).

Xiuping Jia is with the School of Engineering and Information Technology, University of New South Wales, Canberra, ACT 2612, Australia (e-mail: x.jia@adfa.edu.au).

Digital Object Identifier 10.1109/TGRS.2021.3116138

***Index Terms*— Autoencoder, hyperspectral image (HSI) classification, Siamese network.**

I. Introduction

BENEFITING from advances in hyperspectral imaging technology, hyperspectral sensors can image objects in continuous narrow waves, generating a spectral curve that reflects the objects' physical properties. The resultant hyperspectral image (HSI) contains abundant spectral and spatial information about the shape, layout, and context of ground objects. Due to the unique spectral characteristic of ground objects, an HSI can accurately detect, recognize, and analyze ground objects. Compared with multispectral and natural images, HSIs can predict pixel class more accurately. Thus, HSIs have become important tools in monitoring, management, and exploration, and have been widely used in precision agriculture [1], environmental monitoring [2], and skin testing [3].

As an important research topic, HSI classification aims to predict the category of a pixel and has attracted the attention of many researchers in the past several decades. Most published methods can be roughly classified into the following three categories: spectral-based, spatial-based, and spectral–spatial-based. More specifically, spectral features can be raw spectra or their refinements, thus describing the essential characteristics of an object directly. Early on, most researchers focused on methods that considered spectral features and used support vector machine (SVM) [4], logistic regression [5], neural network [6], and random forest [7] classifiers to analyze the spectral feature. However, the redundant information and noise in raw spectra tend to hinder classifier performance. The relationship between the spectra and the ground object is also not linear, making interpretation difficult. Later, many feature extraction methods such as principle component analysis [8], independent component analysis [9], linear discriminant analysis [10], and low-rank [11] were used to eliminate redundant information and noise to produce a more refined representation before classification. Although these methods can improve and enhance the classification accuracy and robustness of a model, the corresponding performance is still unsatisfactory mainly because imaging conditions, including illumination, environmental, and atmospheric factors, tend to cause spectral variability of ground objects. For example, the same ground objects may have different spectral responses, and different

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

ground objects may have the same spectral responses. Thus, spectral-based models with feature engineering basically performed poorly on those samples.

Alternatively, many research studies [12] have shown that the spatial information of the neighbor region could supply the extra spatial structural information to improve model performance. Because neighboring pixels may belong to the same category, the feature of the same class samples may be smoothed, making them more similar. Based on this idea, more methods have used morphological-based [13], [14], filtering-based [15]–[17], and coding-based [18]–[20] strategies to integrate spectral and spatial features. Specifically, morphological-based methods expand the dimensionality of the spatial feature through a series of opening and closing operations. Filtering uses hand-crafted filters such as wavelet and Gabor to eliminate certain frequency information, and the coding scheme fuses geometry information such as edges, corners, and spots by encoding them to determine accurately spatial structure information. Ghamisi *et al.* [21] summarized conventional methods in more detail.

In the last decade, deep learning has achieved significant breakthroughs in many fields such as image classification [22], [23], object detection [24], [25], semantic segmentation [26], [27], and natural language processing [28], [29]. Compared with traditional methods, deep learning methods exhibit strong advantages that do not require complex hand-crafted feature engineering or much prior knowledge. Attributed to the task-related loss function and the learning rule "back-propagation" [30], deep learning can learn discriminative features from data automatically that are more robust than those from traditional methods. Due to their strong ability to learn, deep learning methods have also been used for HSI classification. Specifically, Chen *et al.* [31] used deep learning for HSI classification with an autoencoder. Compared with a fully connected network, a convolutional neural network (CNN) provides local connections, sharing parameters, and shift invariance, making it more flexible to process images. Yu *et al.* [32] constructed a 2-D CNN by stacking multilayer 1×1 convolutional kernels to extract features. However, high spectral dimensionality in an HSI could increase the number of parameters in the 2-D CNN model, while the local spectra correlations can be ignored. Different from 2-D CNN, 3-D CNN models simultaneously consider three dimensions (e.g., height, width, and depth) [33]–[35], which can better represent HSI data. Mei *et al.* [34] constructed a 3-D autoencoder with a 3-D convolutional network to extract the joint spectral–spatial feature. However, a 3-D convolutional kernel typically has a computational cost; thus, Jia *et al.* [36] proposed a lightweight CNN for HSI classification.

In particular, the aforementioned models only have a single branch. Alternatively, many models contain two branches: one is the spectral branch to extract spectral features, and the other is the spatial branch to extract spatial features. Then, the features coming from different branches are fused to form a joint spectral–spatial feature. Generally, a 1-D CNN or fully connected network [37] serves as the spectral branch, and a 2-D CNN [38] serves as the spatial branch. Unlike those models based on a CNN, a recurrent neural network (RNN) can be designed to process sequence data, which can capture the context of an element in the sequence. The spectral information in an HSI can be represented by spectral vectors, and the reflecting value of every band can be considered as an element of the spectral sequence. Thus, an RNN can also be used as a feature module to extract spectral information during HSI classification. Mou *et al.* [39] used the long short-term memory (LSTM) algorithm [40], which is an RNN variant that can capture the long-short dependence of a sequence element, to classify HSI. Later, Liu *et al.* [41] integrated spatial information via neighbor pixel sampling, and Zhou *et al.* [42] used two LSTM modules to extract the spectral and spatial features by converting an image into a sequence.

The success of deep learning is primarily attributed to massive labeled samples, whereas labeling samples in HSI is time-consuming, laborious, and requires prior knowledge. Therefore, how to make deep-learning models work well under small training set scenario is a critical research topic in HSI classification. In the literature, many studies have combined deep learning with other paradigms such as autoencoder [31], [34], [43], transfer learning [38], [44], [45], active learning [33], [46]–[48], and few-shot learning [49], [50], to solve the small training set problem. More precisely, autoencoder is a general method that captures the internal structure information of data from the unlabeled data, which is often combined with a lightweight classifier such as an SVM or logistic regression to classify HSI under small labeled samples. The architecture of autoencoder in HSI classification has evolved from 1-D to 3-D to better fit its data format [31], [34]. Transfer learning aims to transfer knowledge from the source domain to the target domain, decreasing the requirement of massive labeled samples in the latter. It mainly includes two technical lines—fine-tuning and data distribution adaptation. Yang *et al.* [38] designed a deep learning model that is trained on a source dataset and then is fine-tuned on another target dataset. In [45], data distribution adaptation is introduced to make the deep learning model work well even if there are few labeled samples in the target domain.

In general, a lot of labeled samples may be redundant and unnecessary, since there may exist repeating samples providing similar or same information. So how to exploit valuable labeled samples in a dataset is the problem that active learning wants to solve. In HSI classification, most active learning methods [33], [47], [48] are based on posterior probabilities that depend on another classifier to find those that are hard to classify and more valuable. In fact, all the above deep learning models learn the projection between data and task directly. Nonetheless, few-shot learning focuses on learning the difference between data, which is one of the meta-learning. Currently, many few-shot learning-based methods [51]–[55] for HSI classification have been proposed such as prototype network-based and relation network-based. Liu *et al.* [51] utilized a deep learning model to serve as a feature extractor in prototype network. Due to a hand-crafted metric function, the prototype network would not model a complex relationship among samples. To solve this problem, Deng *et al.* [52] introduced relation network into HSI classification. They replace not only feature extractor but

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

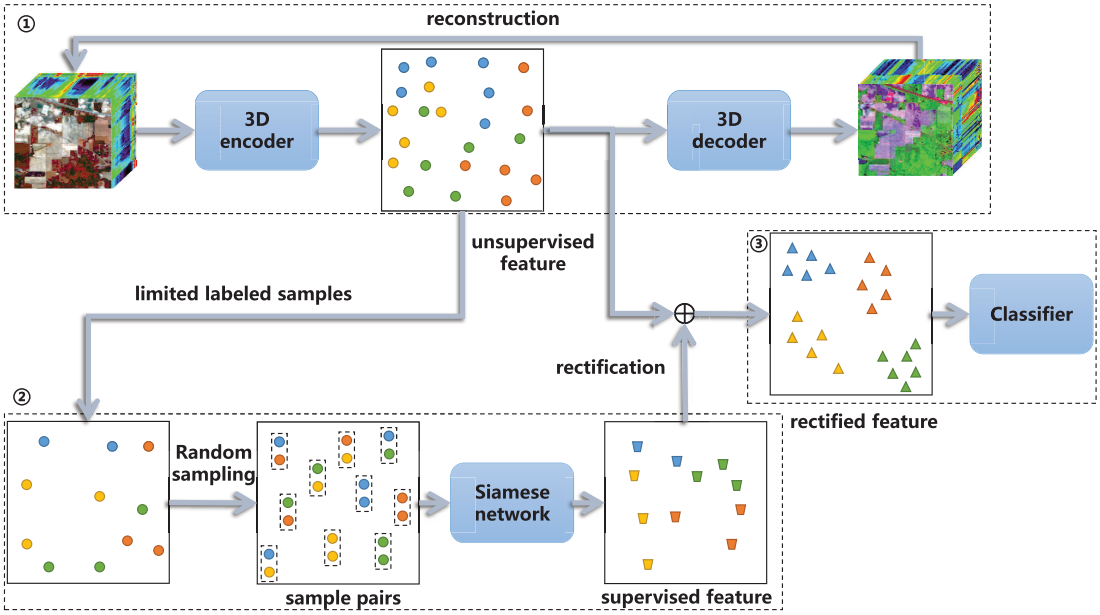


Fig. 1. Flowchart of the proposed 3DAES architecture. In the first step, a 3-D autoencoder is used to learn the HSI's structural information by exploiting a large amount of unlabeled data. In the next step, limited labeled samples composed of sample pairs are used to train the Siamese network with the unsupervised feature. Finally, the rectified feature is classified by logistic regression.

metric module with deep learning. For a comprehensive review of deep learning methods for HSI classification under a small training sample scenario, readers are referred to [56] for more details.

Another way to solve small training samples is data augmentation. Recently, the Siamese network, which aims to measure sample similarity by exploiting the difference between samples, has attracted the attention of researchers in various research fields. Since sample pair construction can augment the number of labeled samples, the Siamese network is a good candidate for a small training set scenario. Generally, a naive sample construction method that traverses all possible combinations in the sample set is adopted to generate sample pairs. However, the increase of labeled sample pairs could aggravate class imbalance and eventually sacrifice classification performance. In addition, generative adversarial network (GAN) [57] is another popular sample augmentation method, which generates fake samples approaching authentic samples. In recent years, GAN with a generator and discriminator has been employed in HSI classification [58], [59] to solve small labeled samples problems. Through repeating adversarial learning, the generator in GAN can generate more approaching authentic samples and the discriminator can discriminate fake samples more accurately. In this progress, the discriminator can extract more and more discriminative features from authentic samples. So after adversarial learning, it can be regarded as a feature extractor module of HSI and fine-tuned on small labeled samples with a classifier. Although these methods based on GAN can gain more samples, it is hard to precisely control the training and convergence procedure.

In this article, we propose a semisupervised Siamese network called 3DAES that integrates autoencoder module and Siamese network to investigate the information in a large amount of unlabeled data and rectify it with a limited labeled sample set, respectively. First, the autoencoder method is trained on a large number of unlabeled samples to learn the refinement representation, creating the so-called unsupervised feature. Second, limited labeled samples are used to train a Siamese network to rectify the unsupervised features to improve feature separability among various classes. Meanwhile, a random sampling scheme is also developed to accelerate training procedures and avoid sample imbalance. The flowchart of the proposed 3DAES architecture is shown in Fig. 1. Experiments with three benchmark HSI datasets consistently demonstrate the effectiveness and robustness of the proposed 3DAES approach with limited labeled samples. The main contributions of this study are summarized as follows.

1) We propose a semisupervised Siamese network, termed 3DAES, for HSI classification. During the training phase of the Siamese network, sample pair construction can augment the training data but with redundancy and less diversity, especially when labeled samples are very limited. On the contrary, unlabeled samples include abundant diversity, which could well compensate for the shortage of the labeled ones. Thus, a semisupervised Siamese network is proposed to integrate both sides. Concretely, an autoencoder module is incorporated, which first learns the requisite data representation directly from the data itself using an unsupervised method. Although the resulting unsupervised feature includes the internal structural information of data, it may lack class separability; thus, the Siamese network is applied to extract the relationship feature with limited labeled samples, which aims to rectify the unsupervised feature to decrease the intraclass distance and also increase the interclass distance. Therefore, the proposed 3DAES approach can fully exploit the unsupervised

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

and supervised information from a large amount of unlabeled data and limited labeled samples, respectively, mitigating any issues that result from a small training set. Unlike the existing semisupervised methods, 3DAES does not fine-tune the unsupervised feature directly instead of rectifying indirectly with small labeled samples. To our best knowledge, this is the first time that the Siamese network is embedded into the semisupervised learning framework in HSI classification.

2) A random sampling strategy is proposed to accelerate model training and avoid prediction bias. This strategy can also be considered as a data augmentation procedure that can make the proposed 3DAES model effective with small training samples. For naive sample construction, the number of negative classes can be much larger than that of positive classes. Under this condition, the classification model generally tends to assign a negative class label to the sample due to its higher gradient contribution during learning. Alternatively, the proposed random sampling scheme is simple but effective at mitigating any imbalance between positive and negative classes, and does not require extra hyperparameters. During every training iteration, the numbers of positive and negative samples are also equal, which markedly accelerates the training procedure.
3) The importance of each module in the proposed 3DAES framework has been verified by a series of ablation experiments, and experiments with three benchmark HSI datasets consistently demonstrate the effectiveness of the proposed method over other methods. The hyperparameters included in the proposed 3DAES framework can be determined by cross-validation, which is kept unchanged in each convolutional layer; thus, the robustness of the proposed 3DAES framework is ensured. For study replication, the code developed for this study can be found at https://github.com/ShuGuoJ/3DAES.git.

The remainder of this article is organized as follows. Section II introduces the basic concepts of autoencoder and Siamese network. A detailed description of the proposed 3DAES model and the sample construction procedure is provided in Section III. In Section IV, comparative experiments and ablation analysis are performed to demonstrate the effectiveness of the proposed model with a small training set. Finally, conclusions are summarized in Section V.

II. Related Works

In this section, related works about autoencoder and Siamese network regarding HSI classification are reviewed to provide a basis for the proposed model.

A. Autoencoder

The architecture of an autoencoder was first proposed by Hinton and Salakhutdinov [26] and consists of two parts: an encoder and a decoder. The former encodes the input $\boldsymbol{x}$ into a hidden representation $\boldsymbol{h}$, and the latter decodes the hidden representation $\boldsymbol{h}$ into an output $\hat{\boldsymbol{x}}$. Usually, the encoder and decoder modules are symmetric and have the same connection pattern. The learning target of the autoencoder is to make the input $\boldsymbol{x}$ and the output $\hat{\boldsymbol{x}}$ similar as much as possible, and can be formulated as follows:

$$\mathcal{D}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = \min |\boldsymbol{x} - \hat{\boldsymbol{x}}|_2 \tag{1}$$

where $\mathcal{D}$ is the similarity function. The hidden representation $\boldsymbol{h}$ would thus carry the internal structural information of the data. Due to the symmetric construction of data, there are many parameters in the overall architecture.

To reduce the number of training parameters, Bengio *et al.* [60] used a greedy layerwise pretraining method to train the autoencoder. They assumed that the first encoder layer's input $\boldsymbol{x}$ is equal to the last decoder layer's output $\hat{\boldsymbol{x}}$. The second encoder layer's input $\boldsymbol{h}_1$ is thus equal to the last decoder layer's output $\boldsymbol{h}_{n-1}$, where n denotes the layer number of the autoencoder. Based on the above hypothesis, we can train each encoder layer and its corresponding decoder layer alone. However, if there is no restriction on the hidden state $\boldsymbol{h}$, the autoencoder may become equivalent to a self-mapping function. There are two variants of autoencoder that can be used to avoid this condition: the sparse autoencoder [61], and the denoise autoencoder [62]. With the sparse autoencoder, normalization or penalty $\Omega(\boldsymbol{h})$ is added to the hidden representation $\boldsymbol{h}$, such as L_1 norm where $\Omega(\boldsymbol{h}) = \sum |\boldsymbol{h}_i|$, to make the largest values tend toward zero and the hidden state $\boldsymbol{h}$ sparser. Unlike the sparse autoencoder, the denoise autoencoder adds a noise $\boldsymbol{x}_e$ to the input $\boldsymbol{x}$, where the real input $\overline{\boldsymbol{x}} = \boldsymbol{x} + \boldsymbol{x}_e$, which allows the encoder to manage some degree of noise.

In HSI classification, Chen *et al.* [31] introduced autoencoder to extract features. To integrate spatial information into the model, the autoencoder flattens the image patch of the central pixel [where the dimensionality is reduced by principal component analysis (PCA)] and concatenates it with the raw spectral vector. The parameters of the encoder and decoder are then used to reduce the model's trainable parameters; the parameter of the decoder $\boldsymbol{\Theta}_d$ is thus the transposed parameter of the encoder $\boldsymbol{\Theta}_e$ (e.g., $\boldsymbol{\Theta}_d = \boldsymbol{\Theta}_e^T$ where T is the transposed operation). Further, many studies based on autoencoder have been proposed in the literature. Xing *et al.* [43] stacked a multilayer denoising autoencoder to extract HSI features while neglecting the spatial feature of the center pixel. Given the strong capability of CNNs for extracting spatial information, later studies combined CNNs to improve model performance. In Yue *et al.* [37] combined autoencoder with CNN to extract spectral and spatial features, respectively. The autoencoder is pretrained with many unlabeled samples, and a standard CNN with a spatial pyramid pooling is used to produce a fixed-length spatial feature. Then the fusion feature that combines the spectral and spatial features is classified by the classifier. The overall model is then fine-tuned via supervised learning. To fuse the spectral and spatial features more accurately, Li *et al.* [63] used a 3-D Gabor operator to preprocess HSI to obtain a joint spectral–spatial feature. Then, an autoencoder was used to transform the joint feature into a more abstract and robust feature. Different from the above autoencoder, Mei *et al.* [34] constructed a 3-D convolutional autoencoder using 3-D CNN. Due to the natural characteristics of the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

3-D convolutional kernel, a 3-D convolutional autoencoder can extract the joint spectral–spatial feature simultaneously without any information loss.

B. Siamese Network

Siamese network [64]–[66] composed of a feature module and a metric module can learn the difference between data by considering a sample pair and determining its similarity. This process distinguishes this method from other learning paradigms. Hadsell *et al.* [67] first proposed a contrastive loss function called ranking loss that trained a CNN to map high dimensional data into a low-dimensional space. The contrastive loss function was then used to group samples in the same class and scatter samples in different classes in a manifold space. The contrastive loss function is formulated as follows:

$$\mathcal{L}_{\text{contrastive}} = \sum yd^2 + (1-y)\max(\text{margin} - d, 0)^2 \quad (2)$$

where y denotes the label of the sample pair. If two samples belong to the same class, the label of the pair is one. Conversely, the label of the sample pair is zero when two samples do not belong to the same class. d represents the distance between two samples. In [67], this function is calculated by the Euclidean distance, and margin means the minimum distance of the interclass. The conservative loss function reduces the distance between samples in the same class and enlarges the distance between samples in different classes until it is larger than margin. To describe distinctions between data more accurately, some studies [64], [65] attempted to find a more complex metric function to replace the Euclidean distance. In [64], Zagoruyko and Komodakis used a variant of the contrastive loss function to train the Siamese network, while the metric between two samples was measured by a fully connected network. For the contrastive loss, determining the appropriate value of *margin* can also be problematic. Unlike the above methods, Han *et al.* [66] transformed a metric problem into a binary classification problem, and the metric module was also replaced by a binary classifier. If samples of the sample pair belong to the same class, the sample pair is positive. Conversely, the sample pair is negative when samples of the sample pair belong to different classes, and the probability distribution of the sample pair represents its matching degree. With the development of deep-learning methods, the Siamese network has been widely used in many fields of study, including facial recognition [68], [69], and visual tracking [70], [71].

Apart from the above fields, the Siamese network has also been used in HSI classification. Liu *et al.* [72] constructed a hybrid model that integrates CNN and RNN to process HSI. The Siamese network was trained with the source dataset via transfer learning to make the model work well with few samples. Then, the overall model is fine-tuned on the target dataset. Since the Siamese network cannot classify HSI directly, a classifier is often needed after training the Siamese network. To unify the training process of Siamese network and classifier into a single scheme, Zhao *et al.* [73] used a multiclassification method with Siamese network to train the network from end to end. During data preprocessing, the method adds a mismatching class represented by y_m to the category. The class y_m means the samples of the sample pair that belong to different classes. If two samples belong to the same class y_i, the label of their sample pair is also y_i. In the inference, the center pixel and its neighboring pixels can generate different sample pairs and the final classification result is obtained by the maximal vote of every sample pair. Rao *et al.* [74] proposed a 3-D CNN with spatial pyramid pooling that adjusts the feature vector length to mitigate information loss after band selection. Meanwhile, a normalization term called hinge loss is imposed to improve model performance. Huang and Chen [75] adapted a two-branch architecture to serve as the feature module of a Siamese network. A metric layer is also used to calculate the similarity of the fusion feature. Miao *et al.* [76] combined autoencoder and Siamese network for HSI classification. However, this study differs from [76] in several critical ways. First, in [76], the autoencoder and Siamese network were trained together via contrastive and reconstruction loss functions, while the autoencoder and Siamese network are trained separately in this study. Since the training procedure of the autoencoder and Siamese network is unified, the structural information inside a large amount of unlabeled data is excluded. Alternatively, in our study, the autoencoder and the Siamese network are separately applied to exploit the unlabeled and labeled data, and thus the generalization ability of the proposed 3DAES approach can be largely improved. Second, the data augmentation strategy proposed in our study can decrease the risk of prediction bias and be more suitable for HSI classification with limited labeled samples. Finally, our 3DAES approach introduced a 3-D CNN module to extract joint spectral–spatial feature from HSI data, guaranteeing the discriminability of the learning features. A detailed description of the proposed 3DAES methodology is presented in Section III.

III. Proposed 3DAES Methodology

To make deep learning function well with limited labeled samples for HSI classification, we propose a semisupervised Siamese network that integrates the advantages of autoencoder framework and Siamese network. The overall architecture of the proposed model is shown in Fig. 1. The autoencoder is trained with a large number of unlabeled samples to extract the unsupervised feature $\boldsymbol{h}_u$, and the Siamese network serves as a relationship function to extract the supervised feature $\boldsymbol{h}_s$ with limited labeled samples from the unsupervised feature. Unlike other methods based on autoencoder, the information inside the labeled data assists in rectifying the unsupervised feature indirectly rather than directly. Then, a simple logistical regression classifier is used to classify the fusion feature. The design of each module is presented in Sections III-A–III-C.

A. 3-D Autoencoder-Based Unsupervised Feature Extraction

Here, the 3-D HSI cube is represented as $\mathbf{X} \in \mathbb{R}^{H\times W\times D}$, where H and W denote the height and width, and D represents the spectral number. Meanwhile, a 3-D neighboring region $\mathbf{X}' \in \mathbb{R}^{H'\times W'\times D}$ of the central pixel at spatial position (i, j)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

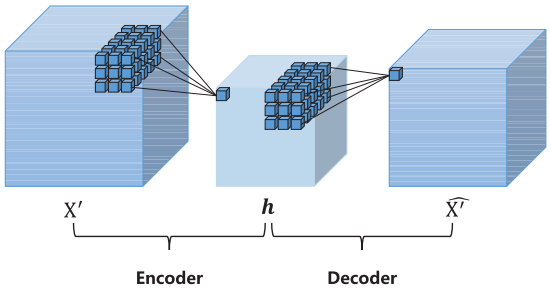


Fig. 2. 3-D autoencoder with two layers. To facilitate exhibiting, a 3-D autoencoder with two layers is picked, while the 3-D autoencoder in our proposed model contains not only two layers. Unlike 2-D CNN, 3-D CNN can describe high, wide, and spectral dimensions in a single frame and thus extract joint spectral–spatial features.

in $\mathbf{X}$ is clipped to form a spectral–spatial combined sample. H' and W' denote the local region size. Thus, the constructed neighboring region dataset is $\mathcal{X} = \{\mathbf{X}'_{i,j}\}$, where $i = 1, \ldots, H$ and $j = 1, \ldots, W$. Autoencoder includes an encoder $f_e(\mathbf{X}')$ and a decoder $f_d(f_e(\mathbf{X}'))$ modules. By minimizing reconstruction error, the hidden representation $\boldsymbol{h}_u = f_e(\mathbf{X}')$ can accompany useful information in the data. To simultaneously create the joint spectral–spatial feature from HSI, a 3-D autoencoder manner is introduced, as shown in Fig. 2. Meanwhile, to avoid self-mapping, a Gaussian noise item $\mathbf{X}_e \in \mathbb{R}^{H' \times W' \times D}$, that follows the standard normal distribution, is taken into consideration. Thus, the real input of autoencoder is $\bar{\mathbf{X}} = \mathbf{X}' + \mathbf{X}_e$. Since different samples exhibit different noise, and each sample has different noise, $\mathbf{X}_e$ is randomly generated during different iteration to accurately model real noise.

On the other hand, the convolutional kernel $\mathbf{K} \in \mathbb{R}^{h_k \times w_k \times d_k \times c_{\text{in}} \times c_{\text{out}}}$, where h_k, w_k, and d_k represent the height, width and the depth of the kernel, and c_{in} and c_{out} denote the input and output channel, respectively, has a great impact on model performance. If $\mathbf{K}$ is too large, CNN may ignore fine-grained features of the object and achieve poor performance. If $\mathbf{K}$ is too small, CNN may slide slowly on the 3-D cube and require more computational time. With low spatial resolution and a small image patch of HSI, we choose a small convolutional kernel $\mathbf{K}$. Concurrently, the output feature channel c_{out} is chosen to reduce the effect of hyperparameters as much as possible. After training, the unsupervised feature $\boldsymbol{h}_u \in \mathbb{R}^{h_{\text{out}} \times w_{\text{out}} \times d_{\text{out}} \times c_{\text{out}}}$ can be gained, where h_{out}, w_{out} and d_{out} are the final height, width, and spectral dimensionality and depend on h_k, w_k, and d_k.

B. Random Sample Pair Generation

To be convenient for the following description, some notations about the training set are introduced. For HSI data, we suppose it has C ground objects and N training samples with each class (it is worth to point out that the equal setting of number of training samples of each class only makes the subsequent discussion easier to be understood. In general, although the number of training samples among various classes can be different, the following derived conclusion still holds). The training data of the cth ($c = 1, \ldots, C$) class is denoted by $\mathcal{X}_c = \{\boldsymbol{x}_{c,1}, \ldots, \boldsymbol{x}_{c,N}\}$ and the corresponding label data by $\mathcal{Y}_c = \{y_{c,1}, \ldots, y_{c,N}\}$. For each $\boldsymbol{x}_{c,n} \in \mathbb{R}^{H' \times W' \times D}, n = 1, \ldots, N$, it corresponds to a local region of the central pixel. Thus, the overall training dataset $\mathcal{A} = \{(\mathcal{X}_1, \mathcal{Y}_1), \ldots, (\mathcal{X}_C, \mathcal{Y}_C)\}$. Meanwhile, for any two samples $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$, $i, j = 1, \ldots, C \times N$, a sample pair $p_{ij} = \{\boldsymbol{x}_i, \boldsymbol{x}_j\}$ is constructed. And its label $\text{label}(p_{ij})$ is formulated as below

$$\text{label}(p_{ij}) = \begin{cases} 0, & y_i \neq y_j \\ 1, & y_i = y_j. \end{cases} \tag{3}$$

Under this naive sample construction method of (3), all possible combinations of the sample pairs are gained, denoted as $\mathcal{P} = \{p_{ij}, \text{label}(p_{ij})\}, i, j = 1, \ldots, C \times N$. Clearly, the number of elements in $\mathcal{P}$ can be computed as

$$I = \frac{1}{2} M(M-1) \tag{4}$$

where M is the number of samples in the training set $\mathcal{A}$. Since there are C classes and N samples for each class, I can be further expressed as

$$I = \frac{1}{2} \text{CN}(CN-1). \tag{5}$$

Concerning the positive sample pair dataset $\mathcal{P}_{\text{pos}}$, which only contains sample pairs with $\text{label}(p_{ij}) = 1$, the number of elements in $\mathcal{P}_{\text{pos}}$ can be calculated as

$$I_{\text{pos}} = \frac{1}{2} \text{CN}(N-1). \tag{6}$$

For the negative sample pair dataset $\mathcal{P}_{\text{neg}}$ that has $\text{label}(p_{ij}) = 0$, the corresponding number of elements I_{neg} can be easily derived as follows:

$$\begin{aligned} I_{\text{neg}} &= I - I_{\text{pos}} \\ &= \frac{1}{2} \text{CN}^2(C-1). \end{aligned} \tag{7}$$

Furthermore,

$$\begin{aligned} \frac{I_{\text{neg}}}{I_{\text{pos}}} &= \frac{\frac{1}{2}\text{CN}^2(C-1)}{\frac{1}{2}\text{CN}(N-1)} \\ &= \frac{N(C-1)}{N-1} \\ &\approx C-1. \end{aligned} \tag{8}$$

It can be clearly seen that I_{neg} is much larger than I_{pos} through the above sample construction. During the training process, if there are no constraints on the model, the proposed method may classify the sample pair as negative with high probability, which increases the gradient. Also, with the increase of the number of training samples within each class N, I would enlarge severely as well, and a big dataset would slow model training at each epoch. To solve the above problems, a random sample pair generation method is proposed to balance the sample amount between the positive and negative classes, thus making $I_{\text{pos}} = I_{\text{neg}}$ and reducing training time. As shown in Fig. 3, for any sample in the training set, a sample is randomly selected from the same class and a different class to, respectively, compose a positive and a negative pair. Here uniform sampling strategy is adopted. Therefore, for each training iteration, $|I_{\text{pos}}| = |I_{\text{neg}}|$ and $|\mathcal{P}| = 2|\mathcal{A}|$ as well, where $|\cdot|$ is the number of elements in the set.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

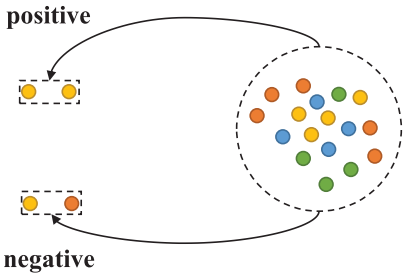


Fig. 3. Random sample pair generation method. For any sample in the training set, a sample is randomly selected from the same class and a different class, respectively, to compose a positive and a negative pair.

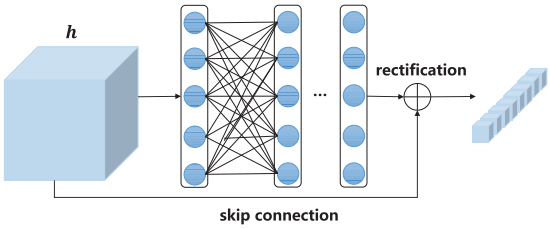


Fig. 4. Rectification module. The supervised feature extracted by the Siamese network is used to guide the unsupervised feature extraction to reduce interclass similarity and increase intraclass similarity.

C. Siamese Network-Based Supervised Feature Extraction

Siamese network takes as an input sample pair p_{ij}, and sample $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ are separately processed. In the following, we make a detailed description of the feature extraction procedure of the Siamese network on sample $\boldsymbol{x}_i$, and the same procedure can be subsequently applied on $\boldsymbol{x}_j$.

After applying 3-D autoencoder on any sample $\boldsymbol{x}_i, i = 1, \ldots, C \times N$, the obtained unsupervised feature $\boldsymbol{h}_u^i = f_e(\boldsymbol{x}_i)$ is a refined representation, but may lack separability. Thus, a well-designed Siamese network with a limited labeled dataset $\mathcal{P}$ is used to improve the feature separability. Behind the 3-D encoder $f_e(\boldsymbol{x}_i)$, a rectification module $f_r(\boldsymbol{h}_u^i)$ is concatenated, as shown in Fig. 4. There is also a skip connection between the input $\boldsymbol{h}_u^i$ and output $\boldsymbol{h}_s^i$ of the rectification module $f_r(\boldsymbol{h}_u^i)$, where

$$\boldsymbol{h}_s^i = f_r\left(\boldsymbol{h}_u^i\right). \tag{9}$$

Thus, the feature module $f_\phi(\boldsymbol{x}_i)$ of the Siamese network consists of an encoder $f_e(\boldsymbol{x}_i)$, a rectification module $f_r(\boldsymbol{h}_u^i)$ and a skip connection. It is worth noting that parameter $\boldsymbol{\Theta}_e$ of $f_e(\boldsymbol{x}_i)$ is fixed after being trained in an unsupervised manner. Here, we suppose that every class has an implicit center point, and the supervised feature $\boldsymbol{h}_s^i$ can make the sample close to its own center point. The fusion manner of features between the rectification module and the skip connection can have different types, and in this study, we consider two simple types: addition and multiplication.

For addition, the output of the feature module $f_\phi(\boldsymbol{x}_i)$ is formulated by

$$f_\phi(\boldsymbol{x}_i) = \boldsymbol{h}_u^i + f_r\left(\boldsymbol{h}_u^i\right). \tag{10}$$

Thus, the rectification module $f_r(\boldsymbol{h}_u^i)$ becomes a residual module, where $f_r(\boldsymbol{h}_u^i) = f_\phi(\boldsymbol{x}_i) - \boldsymbol{h}_u^i$. The residual module was proposed by He *et al.* [23] to avoid degradation of the deep neural network, while the skip connection revises the unsupervised feature $\boldsymbol{h}_u^i$.

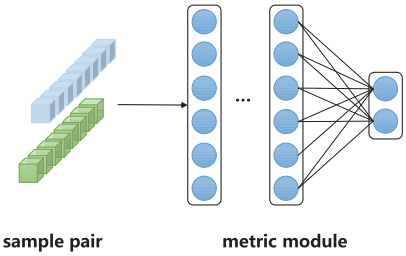


Fig. 5. Binary classifier measures the feature similarity between two samples. One neural unit represents the matching probability, and the other represents the mismatching probability.

For multiplication, the output of the feature module $f_\phi(\boldsymbol{x}_i)$ is formulated by

$$f_\phi(\boldsymbol{x}_i) = \boldsymbol{h}_u^i * f_r\left(\boldsymbol{h}_u^i\right). \tag{11}$$

Thus, the rectification module $f_r(\boldsymbol{h}_u^i)$ becomes an attention module, which is similar to the squeeze-and-excitation module proposed by Hu *et al.* [77]. The output of the rectification module $f_r(\boldsymbol{h}_u^i)$ is an attention vector, which is an important coefficient of the feature that can enlarge the value of interesting features or compress the value of useless features. The attention module has been widely applied in many fields, and many studies have demonstrated its effectiveness.

After the extraction of sample pair features, a metric module is employed to calculate their similarity. To increase the robustness of module hyperparameter, a binary classifier replacing metric module is used to predict the matching probability of the sample pair p_{ij}, as shown in Fig. 5. A cross-entropy loss function is adopted to train the Siamese network, which is described as follows [78]:

$$\mathcal{L} = \sum y_i \hat{y}_i + (1 - y_i)(1 - \hat{y}_i) \tag{12}$$

where $\hat{y}_i$ is the probability belonging to a positive sample. After fine-tuning, a final rectification feature $f_\phi(\boldsymbol{x}_i)$, which is composed with the unsupervised feature $\boldsymbol{h}_u^i$ and rectified by the supervised feature $\boldsymbol{h}_s^i$, can be thus determined. Then, the final rectification features are classified by the logistic regression classifier.

IV. Experiments

In this section, detailed experiments are used to characterize the performance of the proposed 3DAES model with a limited sample set. Three benchmark datasets—University of Pavia (PaviaU), Salinas, and Yellow River Estuary (YRE)—are used to evaluate model performance with overall accuracy (OA), classwise average accuracy (AA), and kappa. OA is defined as the ratio of the classification sample correctly of all samples, where OA $= M'/M$, and M' represents the amount of the corrected classified samples. AA is defined as the average of all-class classification accuracy, which is equivalent to the average of all-class recall, where AA $= \sum M_c'/M_c$ and M_c' and M_c denote correctly classified samples and total samples of the cth class, respectively. Different from OA, AA gives

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LAND COVER CLASSES WITH NUMBER OF SAMPLES PER CLASS FOR THE PAVIAU DATASET

NO.	Class	Train	Test
C1	Asphalt	10	6,621
C2	Meadows	10	18,639
C3	Gravel	10	2,089
C4	Trees	10	3,054
C5	Painted metal sheets	10	1,335
C6	Bare Soil	10	5,019
C7	Bitumen	10	1,320
C8	Self-Blocking Bricks	10	3,672
C9	Shadows	10	937
	Total	90	42,686

TABLE II
LAND COVER CLASSES WITH NUMBER OF SAMPLES PER CLASS FOR THE SALINAS DATASET

NO.	Class	Train	Test
C1	Broccoli green weeds 1	10	1,999
C2	Broccoli green weeds 22	10	3,716
C3	Fallow	10	1,966
C4	Fallow rough plow	10	1,384
C5	Fallow smooth	10	2,668
C6	Stubble	10	3,949
C7	Celery	10	3,569
C8	Grapes untrained	10	11,261
C9	Soil vineyard develop	10	6,193
C10	Corn senesced green weeds	10	3,268
C11	Lettuce romaine 4wk	10	1,058
C12	Lettuce romaine 5wk	10	1,917
C13	Lettuce romaine 6wk	10	906
C14	Lettuce romaine 7wk	10	1,060
C15	Vineyard untrained	10	7,258
C16	Vineyard vertical trellis	10	1,797
	Total	160	53,969

TABLE III
LAND COVER CLASSES WITH NUMBER OF SAMPLES PER CLASS FOR THE YRE DATASET

Class	Class	Train	Test
C1	Building	10	523
C2	River	10	5,366
C3	Salt Marsh	10	4,985
C4	Shallow Sea	10	17,540
C5	Deep Sea	10	18,667
C6	Intertidal Saltwater Marsh	10	2,333
C7	Tidal Flat	10	1,782
C8	Pond	10	1,777
C9	Sorghum	10	636
C10	Corn	10	1,499
C11	Lotus Root	10	2,709
C12	Aquaculture	10	8,009
C13	Rice	10	5,498
C14	Tamarix Chinensis	10	1,210
C15	Freshwater Herbaceous Marsh	10	1,407
C16	Suaeda Salsa	10	864
C17	Spartina Alterniflora	10	570
C18	Reed	10	1,960
C19	Floodplain	10	337
C20	Locust	10	65
	Total	200	77,737

more attention to the class with small samples, emphasizing the importance of classes with fewer samples. kappa is a statistic variant that measures the consistency testing based on the confusion matrix, where kappa $= (p_0 - p_e)/(1 - p_e)$ and p_0 is the overall classification accuracy and p_e is the hypothetical probability of the chance agreement. When the confusion matrix is imbalanced, or AA is small, p_e will be large, yielding a small kappa. Thus, kappa is a more robust metric than OA and AA.

Also, the three datasets investigated in this study have their own characteristics. For example, Salinas has a more regular ground object distribution and sample point, which more effectively smoothes intraclass difference via spatial information. Also, the YRE dataset has a more concentrated ground object distribution and more scattered sample points. The diversity of the three datasets can be useful when evaluating model performance with different scenes and when comparing their differences. Detailed descriptions of each dataset are provided in Section IV-A. In the experiments, we randomly select ten labeled samples from each class to create a training dataset, and the remaining labeled samples are used to create a testing dataset. To mitigate the random impact, sampling is performed ten times; thus, there are ten different training datasets and ten corresponding testing datasets. Finally, the metric of interest is the average of the ten group experiments. There are many factors such as hyperparameters and random initialization of parameters that affect model performance, and it is worth exploring their value before performing a comparative experiment. Concurrently, a series of ablation experiments are performed to evaluate whether the proposed module functions as expected. Finally, several state-of-the-art methods are used for comparison with the proposed model to verify its performance. We also show a final classification map to facilitate the direct comparison of these models' performance.

A. Description of Datasets

1) *PaviaU University (PaviaU):* The PaviaU HSI was photographed over Pavia University by a Reflective Optics System Imaging Spectrometer (ROSIS) sensor with a height and width of 610 and 340, respectively. Every pixel includes 115 bands ranging from 0.43 to 0.86 um. Thus, the HSI data contain 372 100 spatial pixels in total, and 42 776 pixels are labeled in nine categories. The spatial resolution of the HSI is 1.3 m/pixel. After discarding 12 bands that are polluted by noise, the real spectral dimensionality is 103. Detailed class information and their quantities are shown in Table I.
2) *Salinas:* The Salinas HSI was photographed over Salinas Valley, California by an Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) sensor with a height and width of 512 and 217 separately, and 224 bands per pixel ranging from 0.2 to 2.4 μm. Thus, the HSI has a total of 111 104 pixels with 54 129 labeled pixels, which are divided into 16 different classes. The spatial resolution of the HSI is 3.7 m for each pixel, and 20 bands are reflected by water, making them less informative. Thus, the real spectral dimensionality of each pixel is 204. The ground object distribution of Salinas HSI is more regular than that of PaviaU or YRE, and is also easier to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
MODEL SETUP FOR PAVIAU, SALINAS, AND YRE

	hidden unit	neighbor region
PaviaU	128	13×13
Salinas	64	13×13
Yellow River Esturay	128	7×7

classify. Detailed class information and their quantities are shown in Table II.

3) *YRE:* The YRE dataset was captured over the YRE field on November 1, 2018, through a visible shortwave-infrared (SWIR) advanced hyperspectral imager (AHSI). The AHSI sensor is designed and produced by the Shanghai Institute of Techincal Physics, Chinese Academy of Sciences (CAS), and is mounted on China's Gaofen5 satellite platform. Its height and width are 1400, respectively, creating 1 960 000 pixels in total with 77 937 labeled pixels in 20 classes. The spatial resolution of the HSI is 30 m/pixel, and the AHSI sensor collects 330 bands including 150 visible and near-infrared (VNIR) bands and 180 SWIR bands for each pixel. The span of the bands ranges from 0.4 to 2.5 μm. However, there are 280 reserved bands after discarding 50 noisy bands. Compared with the other datasets, the YRE dataset has more samples (both labeled and unlabeled) and more categories. Meanwhile, the low spatial resolution easily generates mixed pixels. The ground object distribution of this HSI is more concentrated than the other HSIs. Detailed class information and their quantities are shown in Table III.

B. Model Setup

Many factors affect model performance, including the depth of the deep-learning model and the number of hidden neural units. In the proposed 3DAES approach, two parameters, both the size of the hidden unit and the neighbor region, are mainly concerned. Evidently, the amount of information considered in 3DAES depends on the hidden-unit quantity. When this quantity is too small, the information stored in hidden neural units decreases; thus, the loss of information during forwarding decreases the classification accuracy of the model. The model then assigns the wrong label to a similar but different pixel due to a lack of discriminating information. Alternatively, if the hidden-unit quantity is too large, the hidden neural units can contain too much information, leading to the increase of information redundancy and the number of trainable parameters, which markedly slow training and inference procedure. Concurrently, redundant information is likely to be noise that hinders model performance, decreasing classification accuracy. To make the matter worse, limited training samples and a large number of trainable parameters inevitably increase the overfitting risk. To facilitate exploring the hidden neural unit size, we hold this value constant in all convolutional layers.

The neighbor region includes spatial information about the center pixel, which is an essential parameter with the model performance. Basically, neighbor pixels of the central one are more likely to belong to the same class; thus the context information of the homogeneous region can smooth the

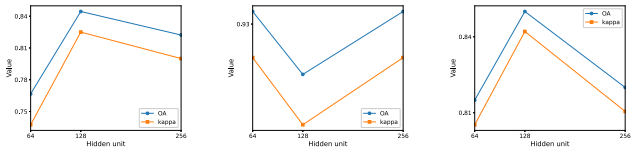


Fig. 6. Sensitivity analysis of hyperparameter (hidden unit) for the proposed 3DAES approach on the three HSIs. (a) PaviaU. (b) Salinas. (c) YRE.

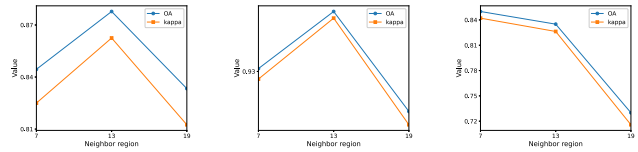


Fig. 7. Sensitivity analysis of hyperparameter (neighbor region) for the proposed 3DAES approach on the three HSIs. (a) PaviaU. (b) Salinas. (c) YRE.

intraclass features and mitigate the phenomenon of different objects with a similar spectrum and the same object with different spectra. For the border pixels, the center pixel is padded by the mirror to extend its neighbor region. When the parameter is too small, the neighbor region information is too limited to smooth the differences between pixels in the same class. Conversely, a larger neighbor region contains pixels from other classes, thereby introducing noise and increasing the difference between pixels in the same class, especially in border pixels. Thus, the size of the neighbor region depends on the ground objects' distribution in the dataset.

In the experiments, the values of the two parameters are decided by ten-fold cross-validation, and the candidate values for hidden unit and neighbor region are 64, 128, and 256, and 7, 13, and 19, respectively. Figs. 6 and 7 visualize the sensitivity analysis of two hyperparameters for the proposed 3DAES approach on the three HSIs.

In Fig. 6, for PaviaU and YRE dataset and when the hidden unit is equivalent to 128, 3DAES achieves the best performance than others. For the Salinas dataset, when values are 64 and 256, they gain similar performance and are best than the 128. Although more hidden units have more powerful learning abilities, it would increase parameter amount at the same time. With respect to a small labeled samples scenario, it would also cause an overfitting phenomenon. Hence, the value of 64 is chosen for the Salinas data.

Generally, a large value of the neighbor region is suitable for the datasets with regular sample distribution to gain more spatial information, and a small value of the neighbor region is suitable for the scattered sample distribution to mitigate introducing noise. From Fig. 7, it can be easily observed that setting neighbor region as 13 is the best for the PaviaU and Salinas datasets. Besides, for the YRE dataset, increasing the size of the neighbor region would decline the overall performance of 3DAES. Thus, the neighbor region for YRE is set as 7. In the end, the chosen values of the two hyperparameters for the three hyperspectral datasets are listed in Table IV.

When training the 3-D autoencoder, 80% of the data is used during training, and the remainder is used for testing. Then, an Adam optimizer is used to optimize the model's parameters

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

TABLE V

PERFORMANCE OF LOGISTIC REGRESSION, 3-D ENCODER, AND 3DAES WITH THE PAVIAU, SALINAS, AND YRE HSIS

Dataset	Method	AA(%)	OA(%)	kappa
PaviaU	Logistic regression	76.29	66.34	0.58
	3D Encoder	81.66	76.64	0.70
	3DAES	**82.97**	**79.63**	**0.74**
Salinas	Logistic regression	90.20	82.89	0.81
	3D Encoder	91.97	86.46	0.85
	3DAES	**94.41**	**90.39**	**0.89**
YRE	Logistic regression	79.23	84.58	0.82
	3D Encoder	83.15	88.03	0.86
	3DAES	**84.97**	**90.02**	**0.88**

with a learning rate of 1e-4, which could mitigate gradient explosion with denoised autoencoder feature. Meanwhile, the training procedure aims to minimize the mean square error (MSE) between the input and output, as shown in (1). Then, the presampling training dataset is transformed into the sample pairs via the random sampling method described in Section III-B. An Adam optimizer with a default learning rate of 1e-3 and a cross-entropy loss function is used to train the Siamese network, as given in (12). To retain the unsupervised feature extracted by the autoencoder, the parameters of encoder module are kept unchanged. Finally, a simple logistic regression is used to classify the rectification feature.

C. Ablation Analysis

1) 3-D Encoder and 3DAES: Compared with the 3-D encoder, our 3DAES proposed a novel way to integrate the unsupervised feature extracted by the autoencoder and the supervised feature extracted by the Siamese network. Different from other methods that transmit the HSI into two different networks to create the unsupervised and supervised features, and then fuse them together, in 3DAES, the supervised feature is extracted after the unsupervised feature, and the unsupervised feature is rectified indirectly to improve feature separability. This operation is performed by the rectification module, as shown in Fig. 4. To verify its effectiveness, the 3-D encoder and 3DAES are trained and tested with the three datasets.

The experimental results of the ablation tests are shown in Table V. The performance of logistic regression on the original HSI data, 3-D encoder, and 3DAES shows a progressive increase across the PaviaU, Salinas, and YRE HSIs, demonstrating the effectiveness of the encoder and rectification module in the proposed 3DAES method. A visualization of the hidden feature is also illustrated in Fig. 8. With the PaviaU HSI, the original sample feature distribution has a high overlapping degree after being projected into the 2-D space. After feature extraction with autoencoder, samples exhibit preliminary separability in the feature space, which shows that autoencoder can refine the data representation by minimizing reconstruction error. Behind the correction by the rectification module, their separability is also clarified. With the Salinas HSI, the blue (Class 15) and brown (Class 8) points are difficult to discriminate, and other sample points of different classes appear mixed. By processing the results from the autoencoder, most points in blue and brown colors could be well separated, and sample points from other classes also exhibit good separability. At last, considering the YRE HSI dataset, after feature extraction by 3-D encoder, a number of pink points remain mixed with gray points in the upper right corner in Fig. 8(f). After rectification, the pink and gray points in the upper right corner in Fig. 8(f) have separated and gathered together. In summary, these quantitative and visualization results constantly demonstrate the effectiveness of the proposed 3DAES method.

TABLE VI

PERFORMANCE OF THE NAIVE SAMPLE CONSTRUCTION AND RANDOM SAMPLE METHOD WITH PAVIAU, SALINAS, AND YRE HSIS

Dataset	Method	AA(%)	OA(%)	kappa
PaviaU	3DAES_naive	79.33	75.64	0.69
	3DAES	**82.97**	**79.63**	**0.74**
Salinas	3DAES_naive	90.24	84.01	0.82
	3DAES	**94.41**	**90.39**	**0.89**
YRE	3DAES_naive	79.88	85.26	0.83
	3DAES	**84.97**	**90.02**	**0.88**

TABLE VII

AVERAGE TRAINING TIME FOR EACH EPOCH OF THE NAIVE SAMPLE CONSTRUCTION AND RANDOM SAMPLE METHOD. WITH THE NAIVE SAMPLE CONSTRUCTION METHOD, EVERY SAMPLE COMBINES WITH THE OTHER SAMPLES TO FORM THE SAMPLE PAIR

	naive	random	ratio
PaviaU	4.60s	1.14s	4.04
Salinas	10.07s	0.76s	13.25
YRE	19.25s	1.02s	18.87

2) Pair Construction Method: Naive sample construction traverses all sample pair combinations during each iteration, increasing training time markedly and producing redundant sample pairs. The naive sample construction method also generates more negative pairs than positive pairs, as discussed in Section III-B. Alternatively, the proposed random sample pair generation method can mitigate this issue. Due to the randomness of sample pair construction, the model may fluctuate during training but can still converge. These issues are similar to those encountered during full- and mini-batch training. To highlight their distinction, we use these sample-pair generation methods to train the model and evaluate its performance, respectively.

The classification result shown in Table VI clearly indicates that the model based on random sampling can achieve superior performance than naive sample construction. Also, This result illustrates that the model based on random sampling can converge even if the sample pair combination keeps changing. Meanwhile, model training time at each epoch is shown in Table VII. Compared with the naive sample construction, random sampling can effectively reduce training time, especially with many labeled samples. With the Salinas and YRE HSIs, the random sampling method decreases training time by ten times compared with the naive sample construction, which is convenient for parameter exploration. Thus, the random sampling method reduces training time and mitigates imbalance between positive and negative pairs, avoiding classification bias and improving classification performance.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

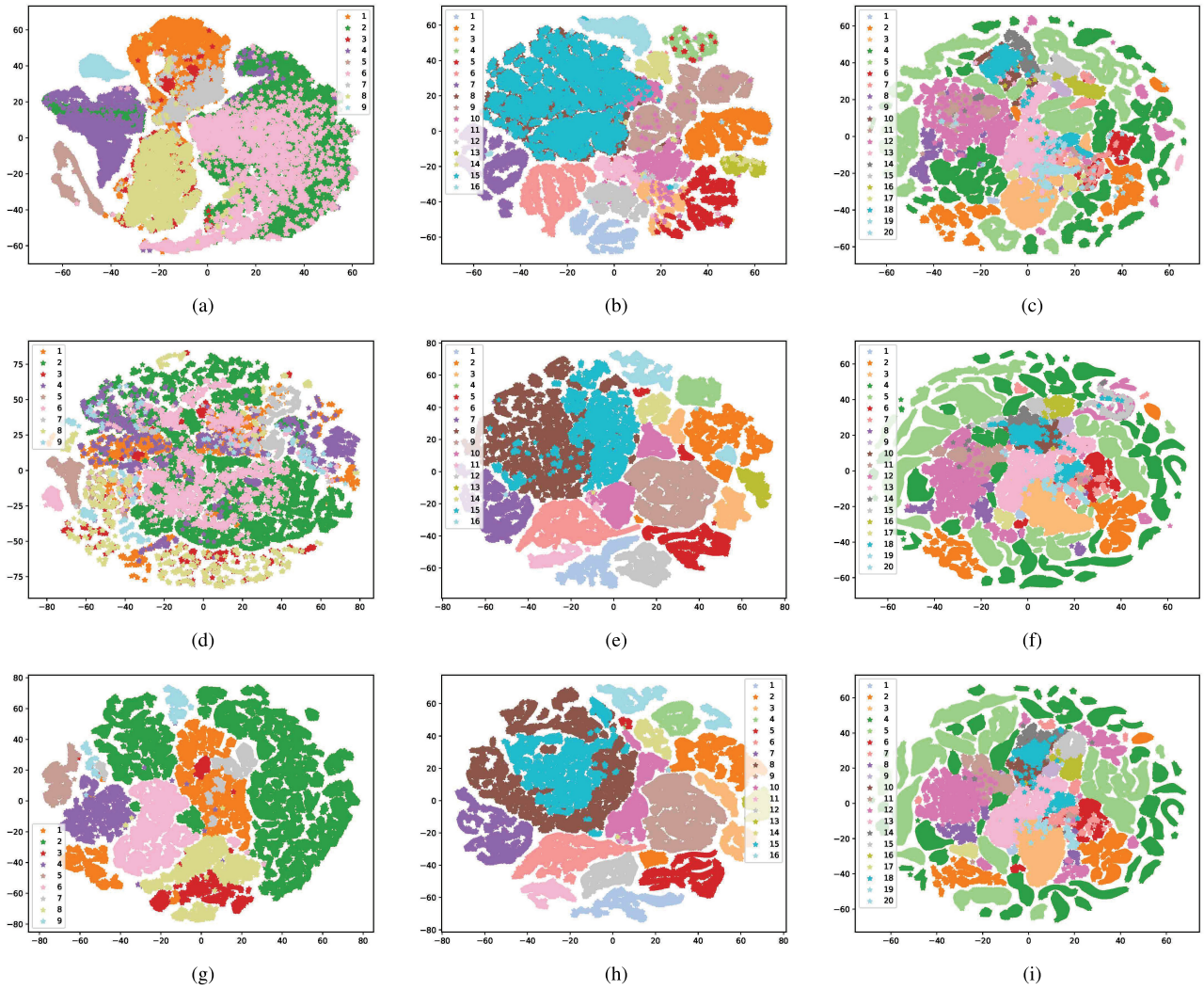


Fig. 8. Sample distribution with the PaviaU, Salinas, and YRE HSIs. Each line denotes the feature distribution of the original, 3-D Encoder, and 3DAES, respectively, and each column denotes the feature distribution with each HSI, respectively. (a) Original features on the PaviaU. (b) Original features on the Salinas. (c) Original features on the YRE. (d) 3-D encoder's features on the PaviaU. (e) 3-D encoder's features on the Salinas. (f) 3-D encoder's features on the YRE. (g) 3DAES's features on the PaviaU. (h) 3DAES's features on the Salinas. (i) 3DAES's features on the YRE.

By comparing Tables V and VI, it can be observed that 3-D Encoder is significantly better than 3DAES_naive. This is reasonable since 3DAES_naive adopts naive sample construction to generate sample pairs, which could easily cause an imbalance between positive samples and negative samples, and further result in classification bias of models. Hence, the class imbalance is an important factor to restrict the model performance, and this is the main reason that we propose a random sample pair generation method to avoid the problem. Moreover, the higher performance of 3DAES than 3-D Encoder and 3DAES_naive have validated the effectiveness of random sample generation.

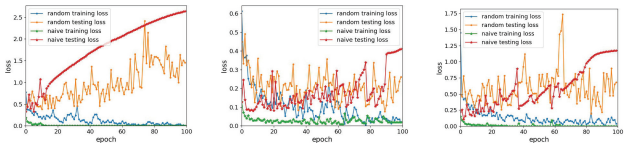
Besides, the learning curves of 3DAES with naive sample construction and random sampling on three benchmark HSI datasets are shown in Fig. 9. As expected, the fluctuation of 3DAES with random sampling is more evident than that with naive sample construction due to the randomness of the sampling scheme. Specifically, the training procedure of 3DAES with naive sample construction converges faster than that with random sampling, but 3DAES with random sampling is capable of mitigating over-fitting. With respect to the PaviaU HSI dataset, naive testing loss rises sharply after few training epochs, while random testing loss maintains in a moderate range for a long time.

TABLE VIII
ABLATION ANALYSIS OF 3DAES WITH DIFFERENT FUSION MANNERS

Dataset	Method	AA(%)	OA(%)	kappa
PaviaU	3DAES_depthwise	74.41	72.03	0.64
	3DAES_mul	80.09	76.56	0.70
	3DAES	**82.97**	**79.63**	**0.74**
Salinas	3DAES_depthwise	92.17	87.34	0.86
	3DAES_mul	93.79	89.18	0.88
	3DAES	**94.41**	**90.39**	**0.89**
YRE	3DAES_depthwise	73.02	79.18	0.76
	3DAES_mul	83.12	88.06	0.86
	3DAES	**84.97**	**90.02**	**0.88**

3) Fusion Type: As discussed in Section II-B, feature fusion can be categorized into two types: addition and multiplication. Under the different fusion types, the rectification module plays

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 9. Learning curves of 3DAES with naive sample construction and random sampling on PaviaU, Salinas, and YRE HSIs. (a) PaviaU. (b) Salinas. (c) YRE.

different roles: in addition, it serves as the residual module; in multiplication, it serves as the attention module. Concurrently, we also use the depthwise separable convolution instead of standard convolution to reduce the number of trainable parameters. The experimental results with the three HSIs are listed in Table VIII. On all datasets, 3DAES outperforms the other two compared models. Although the depthwise separable convolution decouples the standard convolution operation into the spatial transformation and feature channel-wise transformation to reduce the number of trainable parameters, this action degrades model performance. Moreover, 3DAES_mul can amplify interesting features and compress useless features by the attention scores, but it fails to improve the aggregation of the same class samples. For 3DAES, by assuming an implicit cluster center with each class, the difference between the unsupervised feature and the relative cluster center is calculated, which could improve the aggregation of the same class samples and the separability of the different classes.

D. Comparative Experiment

Methods used in comparison with the proposed 3DAES model in this study include SAE_LR [31], Two-CNN [38], 3DVSCNN [33], SSLstm [42], 3DCAE [34], spectral-spatial deep learning (SSDL) [37], graph convolutional network (GCN) [79], semisupervised 1-D GAN (HSGAN) [58], 3-D-GAN [59], and MDL4OW [55]. SAE_LR is an autoencoder that consists of a fully connected network. Before classification, the raw spectral and spatial vectors are preprocessed via PCA and are concatenated to form a joint spectral–spatial vector. To reduce the number of training parameters, a greedy layerwise training method is used to pretrain the model and tie the parameters of the encoder and the decoder modules. Two-CNN is a double branch model that extracts spectral and spatial features, respectively, and then fuses them. To perform well with limited training samples, this method uses transfer learning to pretrain the model, which transfers knowledge from the source data to the target dataset. The source and target domain pairs are Pavia–PaviaU, Indian Pines–Salinas, and Matiwan Village [80]–YRE, respectively. 3DVSCNN is composed of a 3-D CNN that selects informative labeled samples from the training set via active learning. SSLstm is also a double branch architecture but uses an LSTM instead of a CNN, where spectral and spatial information are transformed into sequences separably. 3DCAE is a 3-D CNN autoencoder that is trained with large unlabeled samples, and then an SVM is used to classify the resulting feature. Likewise, SSDL contains double branch as well. The spectral branch is composed of an autoencoder that is pretrained by massive unlabeled samples. After that, the overall architecture is fine-tuned on small labeled samples. Since graph data have a powerful representing ability than grid data, GCN has been introduced for HSI classification. During training, the graph contains not only labeled nodes but also unlabeled nodes. It leverages unlabeled and labeled information, which is a semisupervised method as well. HSGAN is a GAN-based method that continuously generates new fake samples to further improve the discriminator's ability in discriminating authentic or fake samples. Once they have converged, the discriminator is equipped with the power extracting more discriminative and abstract features of HSIs. Then, the labeled samples are utilized to fine-tune the discriminator and a classifier. It is noticed that the last three models are based on semisupervised learning. Unlike HSGAN, 3-D-GAN consists of 2-D CNN that is capable of capturing spectral and spatial information simultaneously. Concurrently, it also incorporates label information during training so that fine-tuning step is unnecessary. MDL4OW involves an encoder followed by a classifier and a corresponding decoder, which integrates intrinsic structured information with labels together in the training phase.

1) Experimental Results on PaviaU: At first, we quantitatively evaluate the performance of 3DAES and the other comparative models in Table IX on the PaviaU dataset. It can be found from Table IX that 3DAES outperforms comparative models on three metrics. Specifically, 3DCAE is the worst, then SAE_LR. 3DCAE and SAE_LR are based on autoencoder, and the difference is that the former adopts a 3-D convolution network and the latter is composed of a general fully connected network. In comparison to the above, 3DAES improves the classification accuracy by a large margin, which implies that the rectification module would enhance the feature representation ability extracted by an autoencoder. The performance of GCN depends on fine graph and node feature construction. Bad graph construction and simple node feature generation would decline classification accuracy. Benefiting from a vast amount of labeled samples in the source domain dataset, Two-CNN gains approximating experimental results to 3DAES. As integrating intrinsic structured information with labels, MDL4OW also achieves good classification performance. 3DVSCNN adopts a 3-D convolution network to extract the spectral and spatial feature simultaneously as well as the active learning paradigm to eliminate the redundant samples, which gains a normal performance. The generating samples in HSGAN can enrich the diversity of dataset and improve generalization, but they severely depend on good initializing parameters and are hard to train. Through adding spatial information, 3-D-GAN is better than HSGAN. Unlike other models, a different deep learning operator – RNN – is utilized in SSLstm. Further, the classification maps of various methods are displayed in Fig. 10. Different from other methods, less salt-and-pepper noise is shown in the 3DAES classification map [Fig. 11(l)]. In addition to 3DVSCNN [Fig. 11(d)] and GCN [Fig. 11(h)], the classification maps obtained by other models have the salt-and-pepper problem. Although 3DVSCNN produces less salt-and-pepper noise, serious classification mistakes exist. For GCN, transforming pixel classification to homogeneous region classification can greatly avoid noise production. However, it would lose much detailed information, especially around

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

TABLE IX

CLASSIFICATION ACCURACY (%) AND KAPPA MEASURE FOR THE PAVIAU DATASET WITH A NUMBER OF ALTERNATIVES

class	SAE_LR	Two-CNN	3DVSCNN	SSLstm	3DCAE	SSDL	GCN	HSGAN	3D-GAN	MDL4OW	3DAES
1	66.67±4.94	72.38±5.55	63.03±16.69	72.59±7.75	49.41±25.25	67.08±8.45	33.63±13.03	68.91±7.09	57.32±9.34	72.77±11.92	**78.49±7.23**
2	56.68±13.23	**87.47±6.12**	69.22±11.43	68.86±16.69	51.60±24.70	73.99±11.20	65.94±10.06	77.77±10.61	80.01±9.76	76.24±9.21	76.80±7.19
3	46.37±17.62	48.44±8.04	**71.77±12.76**	48.08±10.70	54.06±17.95	42.47±19.65	69.91±16.79	39.01±22.37	56.21±11.74	70.39±5.60	64.96±5.33
4	80.10±8.76	**96.50±2.37**	85.10±10.92	79.06±10.95	94.81±4.14	63.67±19.44	59.98±7.50	81.64±7.80	61.57±10.93	93.21±2.98	92.91±3.83
5	98.81±0.80	95.41±5.00	98.61±1.20	93.80±2.76	**99.86±0.19**	99.52±0.38	95.98±9.18	98.46±1.70	86.88±6.91	99.41±0.46	95.27±4.24
6	55.87±16.33	56.50±8.76	75.17±17.09	62.53±14.25	57.40±27.87	49.16±9.93	81.28±6.59	43.00±13.39	76.59±15.82	71.44±13.97	**82.19±9.18**
7	81.42±6.44	62.66±8.59	65.61±23.83	65.39±14.82	80.34±27.24	80.73±11.26	86.41±10.52	83.99±10.87	73.09±17.14	**88.03±8.43**	84.19±8.82
8	66.83±13.15	67.47±6.36	55.77±14.74	67.60±15.38	57.97±22.02	71.72±17.52	52.38±12.06	**79.79±9.75**	73.17±10.04	63.80±12.85	79.13±5.56
9	98.90±1.49	97.35±3.90	96.48±3.38	97.02±5.84	98.76±1.27	99.15±1.42	31.53±13.38	**99.33±1.32**	91.22±6.36	97.47±1.67	92.76±6.47
AA(%)	72.40±1.97	76.02±1.33	75.64±3.38	72.77±4.78	71.58±2.83	71.94±3.11	64.11±2.41	74.66±2.40	72.90±3.31	81.42±2.56	**82.97±1.43**
OA(%)	63.19±4.42	78.20±1.88	70.47±5.65	69.59±6.16	59.14±6.69	69.08±3.97	62.15±3.82	72.17±2.61	73.26±1.73	76.55±4.97	**79.63±2.78**
kappa	0.54±4.31	0.71±2.16	0.63±6.43	0.62±6.58	0.51±5.76	0.60±4.19	0.53±3.67	0.64±2.66	0.65±1.76	0.70±5.65	**0.74±3.11**

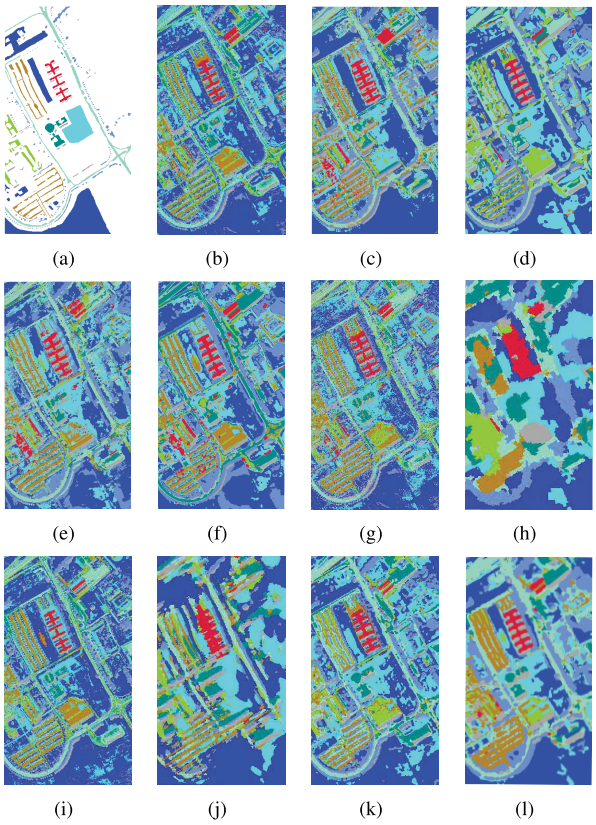


Fig. 10. (a) Ground-truth map and classification maps on the PaviaU dataset by (b) SAE_LR (66.54%), (c) Two-CNN (81.56%), (d) 3DVSCNN (67.63%), (e) SSLstm (71.53%), (f) 3DCAE (61.93%), (g) SSDL (68.66%), (h) GCN (64.93%), (i) HSGAN (73.39%), (j) 3-D-GAN (75.85%), (k) MDL4OW (78.10%), and (l) 3DAES (84.62%).

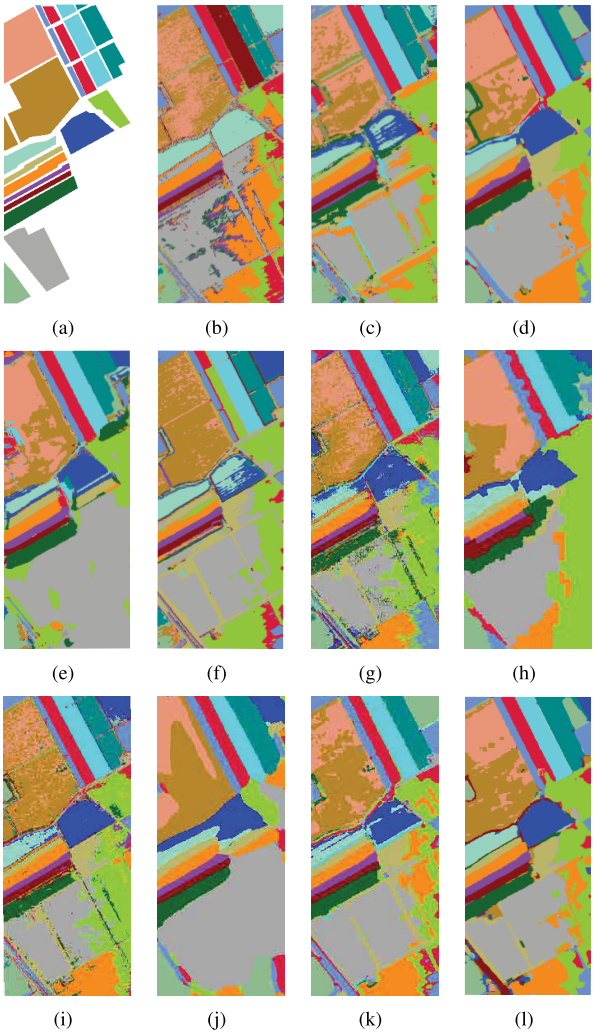


Fig. 11. (a) Ground-truth map and classification maps on the Salinas dataset by (b) SAE_LR (58.17%), (c) Two-CNN (76.67%), (d) 3DVSCNN (87.15%), (e) SSLstm (79.32%), (f) 3DCAE (69.58%), (g) SSDL (81.40%), (h) GCN (85.84%), (i) HSGAN (84.25%), (j) 3-D-GAN (87.05%), (k) MDL4OW (86.44%), and (l) 3DAES (92.21%).

the border [Fig. 11(h)]. These phenomena show that 3DAES can smooth intraclass differences while mitigating salt-and-pepper noise and extracting discriminative interclass features in samples of different classes, validating the effectiveness of the proposed 3DAES method.

2) Experimental Results on Salinas: In Table X, we exhibit class-wise recall and three metrics of 3DAES and comparative methods. Unlike the PaviaU dataset, almost all models have a clear improvement on classification accuracy and kappa, apart from Two-CNN. That can be attributed to the regular distribution of ground objects on the Salinas dataset, which is easy for classification. Concretely, SAE_LR is the worst, then 3DCAE, which also shows the effectiveness of the rectification module. 3DVSCNN has a similar performance to 3DAES. Similarly, the classification maps of the compared ones are shown in Fig. 11, and a similar conclusion to that with the PaviaU dataset can be reached. Therefore, 3DAES has the ability to smooth intraclass features and mitigate salt-and-pepper noise.

3) Experimental Results on YRE: As shown in Table XI, 3DAES gains the highest recall on about three-quarter classes. Concurrently, it is more superior to comparative models. Different from Salinas, the distribution of ground objects on

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

TABLE X

CLASSIFICATION ACCURACY (%) AND KAPPA MEASURE FOR THE SALINAS DATASET WITH A NUMBER OF ALTERNATIVES

class	SAE_LR	Two-CNN	3DVSCNN	SSLstm	3DCAE	SSDL	GCN	HSGAN	3D-GAN	MDL4OW	3DAES
1	86.01±30.85	88.55±7.25	97.92±2.25	79.38±10.17	99.28±0.47	80.18±11.51	94.44±2.77	95.20±2.93	90.43±9.34	86.12±15.94	**99.97±0.05**
2	44.21±40.70	77.06±9.27	**99.71±0.37**	72.49±8.13	59.04±18.59	84.60±9.14	93.11±5.53	97.81±1.92	91.53±9.72	91.97±6.71	99.18±0.85
3	44.72±27.14	73.20±9.13	95.09±4.05	86.83±9.29	66.54±20.73	85.61±13.33	**96.88±2.23**	89.39±7.86	94.49±6.58	80.34±12.33	95.15±3.89
4	97.40±3.43	98.22±2.05	99.28±1.48	**99.45±1.31**	98.65±0.71	98.43±2.03	99.42±1.83	98.74±1.19	98.45±4.31	99.28±0.76	96.47±2.71
5	83.93±18.10	**96.70±1.93**	93.35±3.67	94.95±6.14	81.94±22.11	92.40±6.15	84.75±6.16	94.10±9.65	96.44±3.68	95.06±4.74	91.18±4.29
6	87.28±30.99	99.18±0.48	**99.81±0.33**	93.65±5.52	98.52±0.35	99.29±0.77	97.84±2.75	98.65±1.00	97.10±3.51	99.00±0.94	99.17±0.46
7	96.94±2.74	92.98±2.86	96.73±3.44	87.82±7.32	97.31±1.12	97.81±1.96	94.30±4.03	99.11±0.73	97.65±2.72	98.16±1.47	**99.90±0.09**
8	41.58±32.61	50.66±11.87	67.89±10.51	61.64±8.77	68.11±21.81	73.57±10.57	62.26±6.79	67.79±8.54	75.02±8.27	47.38±22.86	**75.66±6.76**
9	78.45±30.05	83.52±4.58	99.42±1.00	90.47±5.11	95.06±1.86	96.63±3.37	**99.75±0.29**	95.13±3.72	99.50±0.67	97.43±1.81	98.96±1.70
10	30.75±24.52	72.72±7.45	91.75±4.11	86.66±7.92	9.43±4.34	75.62±6.70	84.27±5.33	78.64±6.13	**98.12±2.28**	87.36±5.62	89.74±6.49
11	23.52±34.01	93.24±4.77	95.26±5.73	91.37±6.95	72.26±15.40	91.03±11.09	99.09±1.92	89.98±3.49	**99.88±0.36**	96.16±2.61	99.54±0.57
12	82.63±19.17	93.97±6.96	96.65±6.38	95.38±2.49	72.16±8.90	96.01±5.03	93.69±1.80	98.16±1.07	97.84±1.22	**99.76±0.38**	99.03±1.31
13	92.88±13.51	98.17±2.66	96.64±5.48	96.90±3.50	99.78±0.22	93.85±5.80	85.49±6.42	97.95±1.06	**99.88±0.28**	99.75±0.38	99.76±0.56
14	62.40±36.85	95.07±3.67	99.68±0.59	91.68±5.10	89.93±2.07	88.50±5.42	90.95±7.25	88.91±7.56	**99.85±0.27**	98.10±2.12	97.24±1.44
15	57.10±38.90	60.86±12.21	83.86±12.08	75.55±8.27	56.98±28.09	48.51±20.34	**84.93±9.88**	56.04±10.94	70.00±12.41	77.66±12.35	81.70±8.07
16	76.75±10.20	89.81±4.65	92.03±8.01	88.43±6.33	44.35±8.55	89.50±8.10	95.04±5.80	89.14±8.21	**99.43±1.27**	95.64±3.52	87.94± 8.08
AA(%)	67.91±2.99	85.25±1.42	94.07±1.28	87.04±3.05	75.58±2.04	86.97±1.25	91.01±0.80	89.67±1.11	94.10±1.39	90.57±1.29	**94.41±0.71**
OA(%)	63.26±3.51	76.37±1.49	89.17±1.99	81.20±3.47	72.64±2.14	81.77±1.45	86.19±1.13	83.44±1.57	88.76±1.17	82.44±2.96	**90.39±1.13**
kappa	0.59±3.78	0.74±1.61	0.88±2.19	0.79±3.80	0.70±2.32	0.80±1.65	0.85±1.25	0.82±1.72	0.87±1.31	0.81±3.18	**0.89±1.24**

TABLE XI

CLASSIFICATION ACCURACY (%) AND KAPPA MEASURE FOR THE YRE DATASET WITH A NUMBER OF ALTERNATIVES

class	SAE_LR	Two-CNN	3DVSCNN	SSLstm	3DCAE	SSDL	GCN	HSGAN	3D-GAN	MDL4OW	3DAES
1	41.66±6.64	73.50±9.75	**82.33±5.10**	35.54±12.44	31.53±6.80	51.89±15.46	61.47±8.15	66.20±7.93	40.57±12.95	75.91±6.43	71.01±9.12
2	98.71±1.89	98.78±1.94	95.61±5.49	94.98±5.54	**99.93±0.04**	95.99±7.34	76.61±10.94	98.19±2.85	87.85±9.20	99.25±0.66	98.24±2.22
3	73.74±9.83	75.09±7.67	74.77±11.49	74.92±8.18	69.85±5.84	81.43±7.19	72.49±9.98	**89.63±3.36**	78.03±7.36	81.32±5.16	79.26±3.94
4	62.38±3.91	92.18±4.38	88.07±4.96	76.62±10.42	50.67±13.09	92.65±6.12	87.91±5.09	92.84±2.88	91.14±3.41	89.71±4.13	**95.81±2.15**
5	95.75±0.24	98.08±1.26	96.20±3.00	82.00±13.06	**100.00±0.01**	98.12±0.94	96.66±2.64	97.88±1.34	97.57±2.07	97.55±1.80	97.92±1.15
6	56.61±18.62	75.02±9.16	69.61±9.89	65.22±7.89	75.63±7.09	69.67±8.96	74.03±4.21	71.21±9.36	74.03±14.97	**81.53±7.18**	72.34±9.80
7	43.93±12.63	64.87±4.39	61.31±7.19	53.75±8.11	60.17±17.21	47.79±12.87	49.72±13.01	63.87±6.73	51.36±11.53	64.39±3.82	**70.76±5.18**
8	56.96±7.59	84.63±5.11	88.06±5.16	46.13±26.57	53.47±14.49	78.51±11.05	68.01±10.81	89.81±2.66	64.74±10.56	82.88±5.34	**91.67±3.01**
9	28.44±14.35	83.13±2.08	85.38±4.24	47.30±13.03	45.75±15.27	62.59±11.02	80.31±5.21	78.16±10.03	40.74±12.45	**87.23±4.30**	86.10±2.77
10	47.28±8.29	77.25±9.48	79.63±11.35	52.67±11.10	74.04±19.61	59.01±14.54	69.47±8.44	74.81±9.87	58.42±17.23	79.95±9.12	**83.46±5.41**
11	23.30±9.56	**86.78±12.52**	80.04±15.11	50.39±10.86	43.36±12.09	53.82±12.41	78.44±10.94	71.13±11.27	44.55±13.20	73.57±10.60	79.70±8.37
12	82.72±2.42	74.12±4.66	79.32±6.07	52.03±22.22	69.52±22.69	71.40±6.84	75.45±5.53	**84.24±6.14**	74.34±13.47	78.63±8.66	83.76±5.64
13	82.58±3.36	77.16±4.69	79.21±10.13	63.92±12.86	54.05±16.43	75.52±8.47	74.41±6.41	81.03±7.39	63.51±11.09	81.64±6.91	**86.10±6.00**
14	65.89±10.61	81.31±7.76	77.63±8.52	42.46±12.70	54.55±19.26	84.21±6.42	75.29±7.20	**87.26±7.61**	68.65±14.81	85.45±7.79	85.28±5.88
15	89.55±4.39	**92.75±4.35**	66.86±16.94	61.37±14.69	91.39±2.91	92.71±4.65	62.47±5.79	91.12±8.06	47.97±10.16	85.69±5.19	87.68±5.70
16	91.74±1.18	92.01±2.63	82.25±12.48	65.39±11.79	71.69±14.80	81.47±4.81	77.06±4.68	**94.78±1.90**	78.88±7.75	92.85±1.92	93.73±2.53
17	32.28±8.71	91.68±6.49	73.72±22.07	46.05±13.46	36.77±18.50	72.09±8.59	90.19±5.57	85.16±7.13	87.61±7.31	90.67±7.73	**92.79±8.37**
18	48.92±17.10	62.97±8.20	60.91±13.82	51.22±6.53	24.52±19.61	48.61±11.84	31.17±7.16	68.83±9.34	24.59±11.67	67.67±9.65	**70.91±9.02**
19	67.89±5.29	77.12±3.85	63.80±10.30	59.14±12.59	25.73±16.13	58.78±15.80	62.58±9.19	72.61±8.77	65.10±8.55	79.88±6.61	**81.39±8.94**
20	37.23±17.10	88.92±7.94	**95.69±5.79**	49.85±13.86	86.92±9.74	61.69±9.41	90.15±5.34	71.23±10.88	70.62±14.76	92.46±6.66	91.54±4.12
AA(%)	61.38±2.54	82.37±0.99	79.02±2.84	58.55±7.30	60.98±1.67	71.90±3.11	72.70±1.49	81.50±1.42	65.51±2.46	83.41±1.54	**84.97±1.16**
OA(%)	74.68±1.60	87.08±0.87	84.99±2.01	70.29±7.16	70.58±3.42	83.82±1.42	81.03±1.40	88.84±1.03	79.79±2.34	87.46±1.04	**90.02±0.61**
kappa	0.71±1.83	0.85±1.00	0.83±2.32	0.66±8.35	0.66±4.06	0.81±1.65	0.78±1.58	0.87±1.19	0.77±2.73	0.86±1.21	**0.88±0.71**

YRE is denser and its sample distribution is sparser on the contrary. Although the comparative models have improved as well on YRE, there is still a gap between 3DAES and HSGAN, which provides the second best accuracies. In particular, Two-CNN has a similar performance to 3DAES on PaviaU and 3DVSCNN is on Salinas, while they would not have a better stable performance on different datasets. However, 3DAES is equipped with better stability on three benchmark datasets, which shows its robustness. SSLstm is the worst, instead of SAE_LR and 3DCAE, while SSLstm's performance is approximating to 3DCAE. Nonetheless, 3DAES's performance is much higher than SAE_LR and 3DCAE. According to the visualization of classification maps provided in Fig. 12, we can clearly observe that 3DAES could correctly classify the most spatial pixels in YRE, especially those on the delta area. This indicates that 3DAES can be better to extract discriminative features.

In the following, we reduce the number of training samples to demonstrate the effectiveness of 3DAES. The classification accuracy level as a function of the number of training samples in each class for the interval $(5, \ldots, 10)$ is shown in Fig. 13. As expected, the increase in the number of training samples has a substantial effect on the performance of all the compared methods. Specifically, from Fig. 13, we can see that the curves on the Salinas dataset are more smooth than the other two

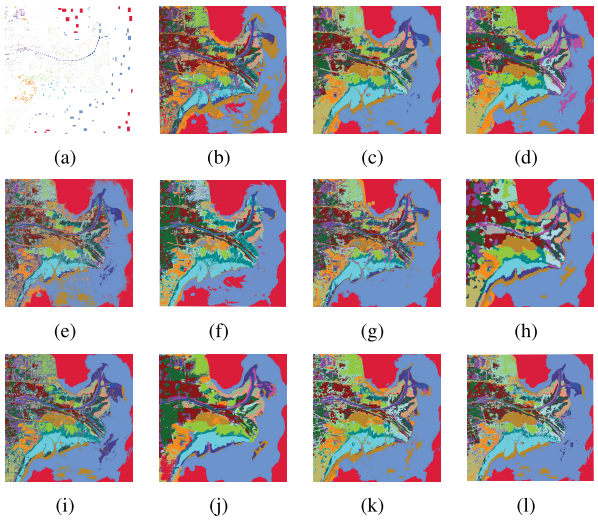


Fig. 12. (a) Ground-truth map, and classification maps on the GF5 dataset by (b) SAE_LR (74.85%), (c) Two-CNN (87.98%), (d) 3DVSCNN (85.10%), (e) SSLstm (76.65%), (f) 3DCAE (76.00%), (g) SSDL (82.59%), (h) GCN (79.26%), (i) HSGAN (88.58%), (j) 3-D-GAN (80.36%), (k) MDL4OW (87.64%), and (l) 3DAES (90.88%).

datasets, which is mainly due to that the data distribution of Salinas is more regular and dense so that introducing extra samples would not have much influence on data diversity. Meanwhile, the performance of the 3DAES is better than that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

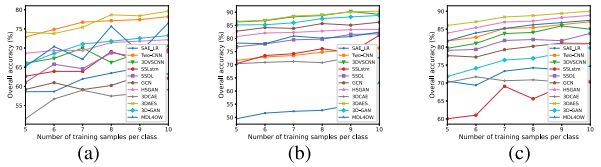


Fig. 13. Classification accuracy versus the different number of training samples per class on the three HSI dataset. (a) PaviaU. (b) Salinas. (c) YRE.

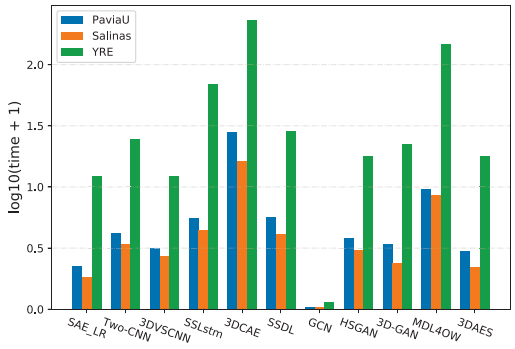


Fig. 14. Total inferring time (seconds) of the compared methods on three benchmark datasets.

of the alternatives in most cases, verifying the efficiency and robustness of the proposed 3DAES approach.

At last, the inferring time of various models is concerned, as illustrated in Fig. 14. All the experiments are carried out on a Tesla P100 graphic processing unit with 16 GB of memory, and the inferring time on full HSI is recorded. Obviously, the inferring time grows with the increase of the spatial size of HSIs. Besides, it can be observed from Fig. 14 that 3DCAE is the slowest one among all models, while GCN is the fastest one as it reduces pixel amount via superpixel segmentation. Besides, the inferring time of our proposed 3DAES model is feasible for HSI data with large spatial coverage.

V. Conclusion

In this article, a semisupervised Siamese network (named 3DAES), that integrates an autoencoder module and a Siamese network to investigate information in a large amount of unlabeled data and rectify it with a limited labeled sample set, has been proposed. This model is composed of three parts, including a 3-D autoencoder, a random sample pair generation module, and a Siamese network. First, the autoencoder method is trained on the massive unlabeled data to learn the refinement representation, creating an unsupervised feature. Second, a random sampling scheme is used to accelerate training and avoid imbalance among various sample classes. Third, based on the unsupervised feature, limited labeled samples are used to train a Siamese network to rectify the unsupervised feature to improve feature separability among various classes.

A series of ablation experiments verify the effectiveness of the rectification module and random sampling strategy in the proposed 3DAES approach. Meanwhile, experimental results on three benchmark datasets that contain different scenes and distributions of ground objects demonstrate the effectiveness and robustness of the proposed 3DAES model. At last, we want to emphasize that the code developed for this study is available at https://github.com/ShuGuoJ/3DAES.git for study replication. Besides, after investigation, the fully connected network used in the rectification module increased the number of trainable parameters of the overall model markedly. Thus, how to design a lightweight and effective rectification module to further improve the performance of 3DAES is a topic for future research, in addition to combine unsupervised and supervised information more effectively.

References

[1] M. Teke, H. S. Deveci, O. Haliloglu, S. Z. Gurbuz, and U. Sakarya, "A short survey of hyperspectral remote sensing applications in agriculture," in *Proc. IEEE 6th Int. Conf. Recent Adv. Space Technol. (RAST)*, Jun. 2013, pp. 171–176.

[2] C. Jänicke, A. Okujeni, S. Cooper, M. Clark, P. Hostert, and S. van der Linden, "Brightness gradient-corrected hyperspectral image mosaics for fractional vegetation cover mapping in northern California," *Remote Sens. Lett.*, vol. 11, no. 1, pp. 1–10, Jan. 2020.

[3] Z. Du, M. K. Jeong, and S. G. Kong, "Band selection of hyperspectral images for automatic detection of poultry skin tumors," *IEEE Trans. Autom. Sci. Eng.*, vol. 4, no. 3, pp. 332–339, Jul. 2007.

[4] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[5] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Semisupervised hyperspectral image classification using soft sparse multinomial logistic regression," *IEEE Geosci. Remote Sens. Lett.*, vol. 10, no. 2, pp. 318–322, Mar. 2013.

[6] Y. Zhong and L. Zhang, "An adaptive artificial immune network for supervised classification of multi-/hyperspectral remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 894–909, Mar. 2012.

[7] S. Jia *et al.*, "Multiple feature-based superpixel-level decision fusion for hyperspectral and LiDAR data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 2, pp. 1437–1452, Feb. 2021.

[8] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear PCA for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–451, May 2012.

[9] A. Villa, J. Chanussot, C. Jutten, J. Benediktsson, and S. Moussaoui, "On the use of ICA for hyperspectral image analysis," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, vol. 4, Jan. 2009, pp. IV-97–IV-100.

[10] S. Jia *et al.*, "Flexible Gabor-based superpixel-level unsupervised LDA for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Jan. 20, 2021, doi: 10.1109/TGRS.2020.3048994.

[11] L. He, J. Li, A. Plaza, and Y. Li, "Discriminative low-rank Gabor filtering for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 3, pp. 1381–1395, Mar. 2017.

[12] A. Plaza, J. Plaza, and G. Martin, "Incorporation of spatial constraints into spectral mixture analysis of remotely sensed hyperspectral data," in *Proc. IEEE Int. Workshop Mach. Learn. Signal Process.*, Sep. 2009, pp. 1–6.

[13] M. D. Mura, A. Villa, J. A. Benediktsson, J. Chanussot, and L. Bruzzone, "Classification of hyperspectral images by using extended morphological attribute profiles and independent component analysis," *IEEE Geosci. Remote Sens. Lett.*, vol. 8, no. 3, pp. 542–546, May 2011.

[14] N. Falco, J. A. Benediktsson, and L. Bruzzone, "Spectral and spatial classification of hyperspectral images based on ICA and reduced morphological attribute profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 11, pp. 6223–6240, Nov. 2015.

[15] S. Jia, L. Shen, and Q. Li, "Gabor feature-based collaborative representation for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 2, pp. 1118–1129, Feb. 2015.

[16] S. Jia, X. Deng, J. Zhu, M. Xu, J. Zhou, and X. Jia, "Collaborative representation-based multiscale superpixel fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7770–7784, Oct. 2019.

[17] S. Jia *et al.*, "Gradient feature-oriented 3-D domain adaptation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Apr. 28, 2021, doi: 10.1109/TGRS.2021.3073699.

[18] W. Li, C. Chen, H. Su, and Q. Du, "Local binary patterns and extreme learning machine for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 7, pp. 3681–3693, Jul. 2015.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

[19] S. Jia, K. Wu, J. Zhu, and X. Jia, "Spectral–spatial Gabor surface feature fusion approach for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1142–1154, Feb. 2019.

[20] S. Jia, Z. Lin, B. Deng, J. Zhu, and Q. Li, "Cascade superpixel regularized Gabor feature fusion for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 5, pp. 1638–1652, May 2020.

[21] P. Ghamisi, M. D. Mura, and J. A. Benediktsson, "A survey on spectral–spatial classification techniques based on attribute profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 5, pp. 2335–2353, May 2015.

[22] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.

[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.

[24] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1440–1448.

[25] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.

[26] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[27] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent. (MICCAI)*, 2015, pp. 234–241.

[28] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, Nov. 2017.

[29] F. Yin, Y.-C. Wu, X.-Y. Zhang, and C.-L. Liu, "Scene text recognition with sliding convolutional character models," 2017, *arXiv:1709.01727*. [Online]. Available: http://arxiv.org/abs/1709.01727

[30] R. Rojas, "The backpropagation algorithm," in *Neural Networks*. Berlin, Germany: Springer-Verlag, 1996, pp. 149–182.

[31] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[32] S. Yu, S. Jia, and C. Xu, "Convolutional neural networks for hyperspectral image classification," *Neurocomputing*, vol. 219, pp. 88–98, Jan. 2017.

[33] L. Hu, X. Luo, and Y. Wei, "Hyperspectral image classification of convolutional neural network combined with valuable samples," *J. Phys., Conf. Ser.*, vol. 1549, Jun. 2020, Art. no. 052011.

[34] S. Mei, J. Ji, Y. Geng, Z. Zhang, X. Li, and Q. Du, "Unsupervised spatial–spectral feature learning by 3D convolutional autoencoder for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6808–6820, Sep. 2019.

[35] S. Jia *et al.*, "3-D Gabor convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Jun. 18, 2021, doi: 10.1109/TGRS.2021.3087186.

[36] S. Jia *et al.*, "A lightweight convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4150–4163, May 2021.

[37] J. Yue, S. Mao, and M. Li, "A deep learning framework for hyperspectral image classification using spatial pyramid pooling," *Remote Sens. Lett.*, vol. 7, no. 9, pp. 875–884, 2016.

[38] J. Yang, Y.-Q. Zhao, and J. C.-W. Chan, "Learning and transferring deep joint spectral–spatial features for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4729–4742, Aug. 2017.

[39] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.

[40] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[41] B. Liu, X. Yu, A. Yu, P. Zhang, and G. Wan, "Spectral–spatial classification of hyperspectral imagery based on recurrent neural networks," *Remote Sens. Lett.*, vol. 9, no. 12, pp. 1118–1127, 2018.

[42] F. Zhou, R. Hang, Q. Liu, and X. Yuan, "Hyperspectral image classification using spectral–spatial LSTMs," *Neurocomputing*, vol. 328, pp. 39–47, Feb. 2018.

[43] C. Xing, L. Ma, and X. Yang, "Stacked denoise autoencoder based feature extraction and classification for hyperspectral images," *J. Sensors*, vol. 2016, pp. 1–10, Jun. 2016.

[44] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[45] Z. Wang, B. Du, Q. Shi, and W. Tu, "Domain adaptation with discriminative distribution and manifold embedding for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 7, pp. 1155–1159, Jul. 2019.

[46] B. Settles, "Active learning literature survey," Dept. Comput. Sci., Univ. Wisconsin-Madison, Madison, WI, USA, Tech. Rep. 1648, 2009.

[47] J. Li, "Active learning for hyperspectral image classification with a stacked autoencoders based neural network," in *Proc. 7th Workshop Hyperspectral Image Signal Process., Evol. Remote Sens. (WHISPERS)*, 2015, pp. 1–4.

[48] Y. Sun, J. Li, W. Wang, P. Antonio, and Z. Chen, "Active learning based autoencoder for hyperspectral imagery classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2016, pp. 469–472.

[49] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2017, pp. 4080–4090.

[50] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1199–1208.

[51] Y. Liu *et al.*, "Deep residual prototype learning network for hyperspectral image classification," in *Proc. 2nd Target Recognit. Artif. Intell. Summit Forum*, vol. 11427, 2020, Art. no. 1142705.

[52] B. Deng, S. Jia, and D. Shi, "Deep metric learning-based feature embedding for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 2, pp. 1422–1435, Feb. 2020.

[53] K. Gao, B. Liu, X. Yu, J. Qin, P. Zhang, and X. Tan, "Deep relation network for hyperspectral image few-shot classification," *Remote Sens.*, vol. 12, no. 6, p. 923, Mar. 2020.

[54] X. Ma, S. Ji, J. Wang, J. Geng, and H. Wang, "Hyperspectral image classification based on two-phase relation learning network," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 10398–10409, Aug. 2019.

[55] S. Liu, Q. Shi, and L. Zhang, "Few-shot hyperspectral image classification with unknown classes using multitask deep learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5085–5102, Jun. 2021.

[56] S. Jia, S. Jiang, Z. Lin, N. Li, M. Xu, and S. Yu, "A survey: Deep learning for hyperspectral image classification with few labeled samples," *Neurocomputing*, vol. 448, pp. 179–204, Aug. 2021.

[57] I. J. Goodfellow *et al.*, "Generative adversarial networks," 2014, *arXiv:1406.2661*. [Online]. Available: http://arxiv.org/abs/1406.2661

[58] Y. Zhan, D. Hu, Y. Wang, and X. Yu, "Semisupervised hyperspectral image classification based on generative adversarial networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 212–216, Feb. 2018.

[59] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5046–5063, Sep. 2018.

[60] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in *Advances in Neural Information Processing Systems*, vol. 19. Cambridge, MA, USA: MIT Press, 2007, p. 153.

[61] A. Coates, A. Ng, and H. Lee, "An analysis of single-layer networks in unsupervised feature learning," *J. Mach. Learn. Res.*, vol. 15, pp. 215–223, Jan. 2011.

[62] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. 25th Int. Conf. Mach. Learn. (ICML)*, 2008, pp. 1096–1103.

[63] J. Li, B. Lorenzo, and S. Liu, "Deep feature representation for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4951–4954.

[64] S. Zagoruyko and N. Komodakis, "Learning to compare image patches via convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 4353–4361.

[65] J. Zbontar and Y. LeCun, "Computing the stereo matching cost with a convolutional neural network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1592–1599.

[66] X. Han, T. Leung, Y. Jia, R. Sukthankar, and A. C. Berg, "MatchNet: Unifying feature and metric learning for patch-based matching," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3279–3286.

[67] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality reduction by learning an invariant mapping," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 2, Jun. 2006, pp. 1735–1742.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.

[68] Z. Bukovčiková, D. Sopiak, M. Oravec, and J. Pavlovičová, "Face verification using convolutional neural networks with Siamese architecture," in *Proc. IEEE Int. Symp. ELMAR*, Sep. 2017, pp. 205–208.
[69] M. Heidari and K. Fouladi-Ghaleh, "Using Siamese networks with transfer learning for face recognition on small-samples datasets," in *Proc. Int. Conf. Mach. Vis. Image Process. (MVIP)*, Feb. 2020, pp. 1–4.
[70] S. Chang, F. Zhang, S. Huang, Y. Yao, X. Zhao, and Z. Feng, "Siamese feature pyramid network for visual tracking," in *Proc. IEEE/CIC Int. Conf. Commun. Workshops China (ICCC Workshops)*, Aug. 2019, pp. 164–168.
[71] X. Chen, X. Zhang, H. Tan, L. Lan, Z. Luo, and X. Huang, "Multi-granularity hierarchical attention Siamese network for visual tracking," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2018, pp. 1–8.
[72] B. Liu, X. Yu, A. Yu, and G. Wan, "Deep convolutional recurrent neural network with transfer learning for hyperspectral image classification," *J. Appl. Remote Sens.*, vol. 12, no. 2, 2018, Art. no. 026028.
[73] S. Zhao, W. Li, Q. Du, and Q. Ran, "Hyperspectral classification based on Siamese neural network using spectral–spatial feature," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2018, pp. 2567–2570.
[74] M. Rao, P. Tang, and Z. Zhang, "A developed Siamese CNN with 3D adaptive spatial–spectral pyramid pooling for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 12, p. 1964, Jun. 2020.
[75] L. Huang and Y. Chen, "Dual-path Siamese CNN for hyperspectral image classification with limited training samples," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 3, pp. 518–522, Mar. 2021.
[76] J. Miao, B. Wang, X. Wu, L. Zhang, B. Hu, and J. Zhang, "Deep feature extraction based on Siamese network and auto-encoder for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2019, pp. 397–400.
[77] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2018, pp. 7132–7141.
[78] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 844–853, Feb. 2017.
[79] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*. [Online]. Available: http://arxiv.org/abs/1609.02907
[80] Y. Cen *et al.*, "Aerial hyperspectral remote sensing classification dataset of Xiongan New Area (Matiwan Village)," *J. Remote Sens.*, vol. 24, no. 11, pp. 1299–1306, 2019.



Sen Jia (Senior Member, IEEE) received the B.E. and Ph.D. degrees from the College of Computer Science, Zhejiang University, Hangzhou, China, in 2002 and 2007, respectively.

Since 2008, he has been with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China, where he is currently a Full Professor. His research interests include artificial intelligence, machine learning, and hyperspectral image processing.



Shuguo Jiang received the B.E. degree in software engineering from Xiamen University of Technology, Xiamen, China, in 2020. He is currently pursuing the master's degree in software engineering with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China.

His research interests include hyperspectral image classification and machine learning.



Zhijie Lin received the B.E. degree from Guangzhou Medical University of Information Management and Information System, Guangzhou, China, in 2017. He is currently pursuing the master's degree in computer technology with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China.

His research interests include hyperspectral image classification and machine learning.



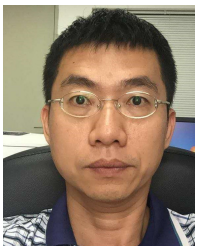
Meng Xu (Member, IEEE) received the B.S. and M.E. degrees in electrical engineering from the Ocean University of China, Qingdao, China, in 2011 and 2013, respectively, and the Ph.D. degree from the University of New South Wales, Canberra, ACT, Australia, in 2017.

She is currently an Associate Research Fellow with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China. Her research interests include cloud removal and remote sensing image processing.



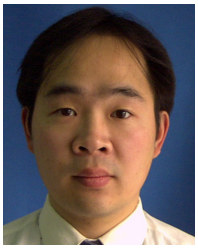
Weiwei Sun (Senior Member, IEEE) received the B.S. degree in surveying and mapping and the Ph.D. degree in cartography and geographic information engineering from Tongji University, Shanghai, China, in 2007 and 2013, respectively.

From 2011 to 2012, he studied with the Department of Applied Mathematics, University of Maryland College Park, College Park, MD, USA, working as a Visiting Scholar with the famous professor John Benedetto to study on the dimensionality reduction of hyperspectral image. From 2014 to 2016, he studied with the State Key Laboratory for Information Engineering in Surveying, Mapping and Remote Sensing (LIESMARS), Wuhan University, Wuhan, China, working as a Post-Doctoral Researcher to study intelligent processing in hyperspectral imagery. He is currently a Full Professor with Ningbo University, Ningbo, Zhejiang, China, and is also working as a Visiting Scholar with the Department of Electrical and Computer Engineering, Mississippi State University, Mississippi, MS, USA. He has authored or coauthored more than 80 journal articles. His research interests include hyperspectral image processing with manifold learning, anomaly detection and target recognition of remote sensing imagery using compressive sensing.



Qiang Huang received the B.S. degree in electrical engineering from Shenzhen University, Shenzhen, China, in 1999, and the Ph.D. degree in electrical engineering and electronics from the University of Liverpool, Liverpool, U.K., in 2004.

He worked as a Research Associate with the University of Leicester, Leicester, U.K., from 2003 to 2004. He was teaching as a Lecturer, an Associate Processor, and a Processor with Shenzhen University. He has authored or coauthored more than 36 research articles. His research interests include distributed embedded real-time systems, digital image processing, super computer systems, and cloud computing.



Jiasong Zhu received the B.E. degree from Huazhong University of Science and Technology, Wuhan, China, in 1997, the M.E. degree from Wuhan University, Wuhan, China, in 2003, and the Ph.D. degree from The University of Hong Kong, Hong Kong, in 2008.

He is currently an Associate Professor with the College of Civil Engineering, Shenzhen University, Shenzhen, China. His research interests include multisensor integration and data fusion, high resolution image processing, and geographic information system (GIS) applications in urban planning and transportation.



Xiuping Jia (Fellow, IEEE) received the B.Eng. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 1982, and the Ph.D. degree in electrical engineering from the University of New South Wales, Canberra, ACT, Australia, in 1996.

Since 1988, she has been with the School of Information Technology and Electrical Engineering, University of New South Wales, where she is currently a Senior Lecturer. She is also a Guest Professor with Harbin Engineering University, Harbin, China, and an Adjunct Researcher with China National Engineering Research Center for Information Technology in Agriculture, Beijing. She has coauthor of the remote sensing textbook titled *Remote Sensing Digital Image Analysis* [Springer-Verlag, third edition (1999) and fourth edition (2006)]. Her research interests include remote sensing, image processing and spatial data analysis.

Dr. Jia is a Subject Editor for the *Journal of Soils and Sediments* and an Associate Editor of the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 09:22:01 UTC from IEEE Xplore. Restrictions apply.